Data Management with R

Noushin Nabavi

2026-05-24

Contents

Welcome

This book provides practical teaching materials and examples for **data management and reproducible analysis using R**. The focus is on building clear, structured, and repeatable workflows for working with data in an applied setting.

The materials are designed for learners who are developing confidence with R, RStudio Projects, the tidyverse, R Markdown, and bookdown. The examples use instructional datasets and emphasize the process of preparing, cleaning, joining, reshaping, visualizing, and communicating data.

Overview

This book helps learners:

- Develop practical data-wrangling skills using the tidyverse
- Apply reproducible project structures for consistent analysis
- Work with tidy data principles
- Read, clean, reshape, and join datasets
- Use strings, dates, and regular expressions in data cleaning
- Create clear tables, figures, and analysis outputs
- Communicate results through R Markdown, bookdown, dashboards, and storyboards

The goal is not advanced statistical modelling. Instead, the emphasis is on building reliable workflows that support clear, organized, and reproducible analysis.

How to Use This Book

Each chapter introduces a data management topic and includes worked examples, code chunks, explanations, and practice activities. Learners are encouraged to:

- Read each section carefully
- Run the code chunks in RStudio
- Modify examples to test their understanding
- Keep raw data unchanged
- Save generated outputs in the `Outputs/` folder
- Use clear file names and reproducible project structure

The chapters are intended to be followed in order, but individual chapters can also be used as standalone reference materials.

Recommended Chapter Flow

The book is organized into the following learning sequence:

1. **Project Setup**
   Set up an RStudio Project, organize folders, and understand reproducible workflows.
2. **Tidyverse Basics**
   Learn core tidyverse functions for reading, selecting, filtering, mutating, and summarizing data.
3. **Joining Data**
   Practice combining datasets using keys and joins, with examples from the Titanic dataset.
4. **Data Cleaning**
   Work with raw files, column names, missing values, reshaping, and structured cleaning steps.
5. **Strings and Regular Expressions**
   Use `stringr`, `separate()`, `str_extract()`, and regex patterns to clean and parse text data.
6. **Data Visualization**
   Create clear visual summaries using `ggplot2`, including line plots, bar charts, and grouped visualizations.
7. **Exploratory Data Analysis**
   Build an organized EDA workflow using summaries, plots, missing-value checks, and data validation.
8. **Communicating Results**
   Use storyboards, dashboards, and reporting structure to communicate data insights clearly.

Project Structure

A recommended project structure is shown below:

```
project/

  index.Rmd
  01_project_setup.Rmd
  02_tidyverse_basics.Rmd
  03_joining_data.Rmd
  04_data_cleaning.Rmd
  05_strings_regex.Rmd
  06_visualization.Rmd
  07_tidyverse_analysis.Rmd
  08_storyboard.Rmd

  _bookdown.yml
  _output.yml
  style.css
  data_management.Rproj

  Raw Data/
  Outputs/
  docs/
  archive/

  README.md
  LICENSE
  .gitignore
```

Raw data should remain in `Raw Data/`. Files created by analysis should be saved in `Outputs/`. Rendered book files can be saved in `docs/` if the book is being published with GitHub Pages.

Required R Packages

The book uses several R packages, including:

```
install.packages(c(
  "bookdown",
  "tidyverse",
  "ggplot2",
  "readxl",
  "lubridate",
  "stringr",
  "purrr",
  "knitr",
  "kableExtra",
  "here",
  "flexdashboard"
))
```

Load commonly used packages with:

```
library(tidyverse)
library(ggplot2)
library(readxl)
library(lubridate)
library(stringr)
library(purrr)
```

Building the Book

To build the book from RStudio, open the project file:

```
data_management.Rproj
```

Then use the **Build Book** option in the Build pane.

You can also render the book from the R console:

```
bookdown::render_book("index.Rmd")
```

Data and Reproducibility Notes

This project uses relative file paths and a structured folder system. This makes the work easier to share, rerun, and maintain.

Recommended practices:

- Do not edit raw data files directly
- Keep all raw datasets in `Raw Data/`
- Save processed files in `Outputs/`
- Use clear and consistent file names
- Use RStudio Projects to keep paths stable
- Comment code where needed
- Keep analysis steps in a logical order

All datasets used in the course are simulated, fictitious, or instructional only. They are intended for teaching data management concepts, not for drawing real-world research conclusions.

Intended Audience

This book is intended for learners who have basic familiarity with R and RStudio and want to build stronger applied data management skills. It can be used for:

- Self-guided learning
- Training workshops
- Teaching support materials
- Practice exercises
- Templates for reproducible workflows

Licence

For educational use. See `LICENSE` for details.

Chapter 1

Project Setup and Reproducible Workflows

A well-organized project structure is one of the foundations of reproducible data analysis. In real-world projects, data files, scripts, figures, notes, reports, and outputs can quickly become difficult to manage if they are not organized consistently. Poor organization often leads to broken file paths, duplicated work, missing outputs, and confusion about which version of a file should be used.

This chapter introduces a simple workflow for organizing R projects using RStudio Projects, relative file paths, R Markdown, and bookdown. The goal is not only to keep files tidy, but also to make analyses easier to reproduce, easier to review, and easier to share with others.

By the end of this chapter, you should be able to create a structured RStudio project, organize files into logical folders, work with relative file paths, and understand the role of Git and GitHub in reproducible workflows.

1.1 Organizing a reproducible project

A typical data analysis project includes several different types of files. These often include raw datasets, cleaned datasets, R scripts, figures, tables, notes, and reports. Without a clear structure, projects can quickly become difficult to navigate and maintain.

A good project structure helps you keep related files together and reduces the likelihood of errors. It also makes collaboration easier because other users can understand the workflow more quickly.

For this course, all work should be placed inside a single main project folder. Within that folder, separate folders should be created for raw data, generated outputs, reports, and archived material.

A recommended structure is shown below.

```
Data-Management-with-R/

   data_management.Rproj
   index.Rmd
   01_project_setup.Rmd
   02_tidyverse_basics.Rmd
```

```
03_joining_data.Rmd
04_data_cleaning.Rmd
05_strings_regex.Rmd
06_visualization.Rmd
07_exploratory_analysis.Rmd
08_storyboard_reporting.Rmd

Raw Data/
Outputs/
docs/
archive/

_bookdown.yml
_output.yml
style.css
README.md
LICENSE
.gitignore
```

The `.Rproj` file allows RStudio to recognize the project and automatically set the correct working directory. The `.Rmd` files contain the book chapters, while the `Raw Data/` folder stores original datasets that should remain unchanged throughout the analysis process.

The `Outputs/` folder should contain generated files such as cleaned datasets, tables, and figures. The `docs/` folder stores the rendered bookdown website, while the `archive/` folder can be used for old drafts or unused material that should not be deleted permanently.

The `_bookdown.yml` file controls the order of chapters and output settings, and `_output.yml` controls formatting options for the final rendered book.

One of the most common beginner mistakes in R is working with files scattered across different folders. Keeping all project files inside a single RStudio Project helps avoid many file path and working directory problems.

1.2 Working with RStudio Projects and file paths

An RStudio Project provides a dedicated working environment for a single analysis or course. Instead of opening individual files manually from different folders, the entire project can be opened through the `.Rproj` file.

To create a new project in RStudio, select **File > New Project** and either create a new directory or connect to an existing folder. Once the project is created, opening the `.Rproj` file automatically sets the project folder as the working directory.

Using projects becomes especially important when working with file paths. Many beginners use absolute file paths that only work on their own computer. For example:

```
read_csv("C:/Users/YourName/Desktop/Data/Population_Estimates.csv")
```

Although this works locally, the path will fail on another computer because the directory structure is different.

Instead, use relative paths that begin from the project folder:

```
read_csv("./Raw Data/Population_Estimates.csv")
```

Relative paths make projects portable and reproducible because the code can run correctly on different computers as long as the project structure remains the same.

Another useful approach is to use the `here` package:

```
library(here)

read_csv(
  here("Raw Data", "Population_Estimates.csv")
)
```

Both approaches are acceptable, but the important idea is to avoid hard-coded computer-specific paths whenever possible.

1.3 Naming files and managing outputs

Consistent file names make projects easier to understand and maintain. File names should be short, descriptive, and predictable. Spaces are usually avoided because they can create problems in some programming environments, so underscores are preferred instead.

Examples of clear file names include:

```
01_project_setup.Rmd
02_tidyverse_basics.Rmd
cancer_rates.csv
air_quality_summary.csv
```

Avoid vague names such as:

```
final_version2_revised_NEW.Rmd
```

As projects grow, unclear naming conventions can make it difficult to determine which files are current and which files are outdated.

It is also important to separate raw data from generated outputs. Raw datasets should remain unchanged so that the original source data is always preserved. Cleaned datasets, transformed files, tables, and figures should instead be written to the `Outputs/` folder.

For example:

```
rates_clean <- rates %>%
  filter(letter == "C")

write_csv(
  rates_clean,
  "./Outputs/cancer_rates_clean.csv"
)
```

This workflow creates a clear separation between original data and processed outputs, which improves reproducibility and transparency.

Avoid manually editing files inside the `Raw Data/` folder. If data cleaning is required, create a cleaned version in the `Outputs/` folder instead.

1.4 R Markdown and bookdown workflows

R Markdown combines text, code, and results within a single document. This makes it possible to explain the purpose of an analysis while also showing the code and outputs used to produce the results.

In this course, each major topic is stored in a separate `.Rmd` file, and bookdown combines these files into a single website or book.

The order of chapters is controlled by `_bookdown.yml`. A typical configuration looks like this:

```
book_filename: "data-management-with-r"
output_dir: "docs"

rmd_files:
  - "index.Rmd"
  - "01_project_setup.Rmd"
  - "02_tidyverse_basics.Rmd"
  - "03_joining_data.Rmd"
  - "04_data_cleaning.Rmd"
  - "05_strings_regex.Rmd"
  - "06_visualization.Rmd"
  - "07_exploratory_analysis.Rmd"
  - "08_storyboard_reporting.Rmd"
```

Inside each chapter file, only one top-level heading should normally be used:

```
# Chapter Title
```

Subsections should use lower heading levels:

```
## Section
### Subsection
```

Using multiple # headings inside the same file causes bookdown to incorrectly interpret them as separate chapters, which leads to a disorganized sidebar and navigation structure.

A typical R Markdown workflow often follows a consistent sequence:

1. Introduce the purpose of the analysis.
2. Load the required packages.
3. Read the data.
4. Clean or transform the data.
5. Create summaries, tables, or figures.
6. Explain the results.
7. Save outputs if needed.

This structure helps readers understand not only what the code does, but also why each step is being performed.

1.5 Git, GitHub, and reproducible analysis

As projects become larger and more collaborative, version control becomes increasingly important. Git is a version control system that tracks changes made to files over time, while GitHub provides an online platform for storing and sharing Git repositories.

Using Git and GitHub allows you to maintain a history of your work, recover earlier versions of files, collaborate with others safely, and create a backup of your project outside your local computer.

RStudio includes built-in Git support, allowing Git operations to be performed directly within the RStudio interface.

The recommended workflow is:

1. Create a GitHub repository.
2. Clone the repository into an RStudio Project.
3. Edit files locally.
4. Commit changes regularly.
5. Push updates back to GitHub.

For detailed guidance, the online resource *Happy Git with R* provides an excellent introduction:

https://happygitwithr.com/

Git and GitHub are strongly recommended for reproducible workflows, but they are not mandatory for completing this course. You can still complete all workshop activities locally using RStudio Projects and bookdown.

1.6 Chapter summary

In this chapter, you learned how to organize a reproducible R project using RStudio Projects, relative paths, R Markdown, and bookdown. You also learned how to separate raw data from generated outputs, organize chapter files consistently, and structure projects in a way that improves reproducibility and collaboration.

A clean project structure makes analyses easier to understand, easier to maintain, and easier to share with others. As projects grow larger and more complex, good organizational practices become increasingly important.

Chapter 2

Tidyverse Basics

The **tidyverse** is a collection of R packages designed for data science and data analysis. These packages provide a consistent and readable approach for importing, cleaning, transforming, visualizing, and analyzing data. In practice, the tidyverse is widely used because it allows complex workflows to be written in a clear and organized way.

In this chapter, you will learn how to load tidyverse packages, read datasets into R, inspect data frames, and perform common data manipulation tasks using `dplyr`. By the end of the chapter, you should feel comfortable performing basic exploratory data analysis and creating simple data transformation pipelines.

2.1 Loading packages and reading data

Before working with data in R, the required packages must first be loaded. The `tidyverse` package includes several commonly used packages such as `dplyr`, `ggplot2`, `readr`, and `tidyr`.

```
library(tidyverse)
```

In most projects, the first analytical step is reading raw data into R. Throughout this course, raw datasets should be stored inside the `Raw Data/` folder introduced in the previous chapter.

The example below demonstrates how to read a `.csv` file using `read_csv()` from the `readr` package.

```
example_data <- read_csv(
  "./Raw Data/example.csv"
)
```

Using relative paths helps keep projects reproducible because the code can run correctly on different computers as long as the project structure remains consistent.

One of the most common beginner problems in R involves incorrect working directories or broken file paths. Using RStudio Projects and relative paths helps avoid many of these issues.

2.2 Inspecting data frames

After importing a dataset, it is important to understand its structure before beginning any analysis. This step is often called *data inspection* or *exploratory inspection*.

Several functions are useful for quickly examining a data frame.

```
glimpse(example_data)

summary(example_data)

dim(example_data)

names(example_data)
```

The `glimpse()` function provides a compact overview of the dataset structure, including variable names and data types. The `summary()` function generates summary statistics for each variable, while `dim()` returns the number of rows and columns in the dataset. The `names()` function lists all variable names.

Inspecting data early in the workflow helps identify potential issues such as missing values, incorrect variable types, or unexpected column names.

2.3 Data manipulation with dplyr

One of the most powerful features of the tidyverse is the `dplyr` package, which provides functions for manipulating and transforming data frames.

Data manipulation workflows are commonly built using the pipe operator `%>%`. The pipe sends the output of one step directly into the next step, making code easier to read and understand.

The example below demonstrates several commonly used `dplyr` verbs.

```
example_data %>%

  select(column_1, column_2) %>%

  filter(column_1 == "value") %>%

  mutate(new_column = column_2 * 2) %>%

  group_by(column_1) %>%

  summarize(
    total = sum(new_column, na.rm = TRUE)
  )
```

In this workflow:

- `select()` chooses specific columns from the dataset.
- `filter()` keeps only rows that satisfy a condition.
- `mutate()` creates a new variable.
- `group_by()` creates groups within the data.
- `summarize()` calculates summary statistics for each group.

Together, these functions form the foundation of many data analysis workflows in R.

The pipe operator `%>%` can be interpreted as "then." Reading pipelines from top to bottom often makes the analysis logic easier to follow.

2.4 Practice exercise

To reinforce the concepts introduced in this chapter, try completing the following short exercise using one of your own datasets or a practice dataset provided in the course materials.

1. Import a `.csv` dataset from the `Raw Data/` folder.
2. Use `glimpse()` and `summary()` to inspect the dataset.
3. Select several variables of interest.
4. Filter the data to one category or group.
5. Create a new variable using `mutate()`.
6. Calculate a simple summary statistic using `summarize()`.

As you practice these steps, focus not only on writing code that works, but also on creating workflows that are readable, organized, and reproducible.

2.5 Chapter summary

In this chapter, you were introduced to the tidyverse and several core tools used in modern R workflows. You learned how to load packages, import datasets, inspect data frames, and manipulate data using common `dplyr` functions.

These foundational skills will be used throughout the remainder of the course for data cleaning, visualization, exploratory analysis, and reporting.

Chapter 3

Joining Data

In many real-world projects, data are stored across multiple datasets rather than in a single table. For example, one file may contain patient demographic information, while another contains laboratory results or medication records. To perform meaningful analysis, these datasets often need to be combined into a single data frame.

The process of combining datasets based on one or more shared variables is called a *join*. In the tidyverse, joins are commonly performed using functions from the `dplyr` package. Understanding how joins work is an essential skill in data management because incorrect joins can easily introduce duplicate rows, missing information, or inaccurate results.

In this chapter, you will learn how to identify join keys, understand common join relationships, and use functions such as `left_join()` and `inner_join()` to combine datasets safely and effectively.

3.1 Understanding join keys

A join works by matching rows between two datasets using one or more shared variables called *keys*. A key is typically an identifier such as a patient ID, employee number, postal code, or year.

The example below creates two small datasets that share a common variable called `key`.

```
library(tidyverse)

x <- tribble(
  ~key, ~val_x,
     1, "x1",
     2, "x2",
     2, "x3",
     1, "x4"
)

y <- tribble(
  ~key, ~val_y,
     1, "y1",
```

```
    2, "y2"
)
```

In this example, the variable `key` is used to connect the two tables. Dataset `x` contains multiple rows for some keys, while dataset `y` contains only one row for each key.

Before performing joins, it is important to understand the relationship between the datasets. Common relationships include:

- one-to-one relationships, where each key appears only once in both tables;
- one-to-many relationships, where one table contains repeated keys; and
- many-to-many relationships, where both tables contain duplicate keys.

Understanding these relationships is important because joins involving duplicate keys can unexpectedly increase the number of rows in the final dataset.

3.2 Performing joins with dplyr

One of the most commonly used joins is the `left_join()`. A left join keeps all rows from the first dataset and adds matching information from the second dataset whenever a match is found.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##     key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     2 x3    y2
## 4     1 x4    y1
```

In this example, every row from dataset `x` is preserved. The values from `y` are added based on matching values of `key`.

Another commonly used join is the `inner_join()`, which keeps only rows that have matches in both datasets.

Different join functions are useful for different analytical situations:

- `left_join()` keeps all rows from the left table;
- `inner_join()` keeps only matched rows;
- `right_join()` keeps all rows from the right table;
- `full_join()` keeps all rows from both tables; and
- `anti_join()` identifies rows that do not have matching values in another dataset.

In practice, `left_join()` is often the safest starting point because it preserves the structure of the main dataset while adding additional variables from another table.

When performing joins, always think carefully about which dataset should be considered the “main” table. The order of tables in a join can affect the final result.

3.3 Checking for duplicate keys

Before joining datasets, it is good practice to verify whether the join key is unique. Duplicate keys are one of the most common causes of unexpected results during data merging.

The following code counts the number of times each key appears in both datasets.

```
x %>%
  count(key) %>%
  filter(n > 1)
```

```
## # A tibble: 2 x 2
##     key     n
##   <dbl> <int>
## 1     1     2
## 2     2     2
```

```
y %>%
  count(key) %>%
  filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: key <dbl>, n <int>
```

In dataset `x`, some keys appear more than once, while dataset `y` contains unique keys. This means the join represents a one-to-many relationship.

If both datasets contain duplicate keys, the join may create many-to-many relationships, which can dramatically increase the number of rows in the output dataset.

Always inspect duplicate keys before performing joins on large datasets. Unexpected duplication is one of the most common data management errors in real-world projects.

Another useful function is `anti_join()`, which identifies rows in one dataset that do not have matching values in another dataset.

For example:

```
anti_join(x, y, by = "key")
```

This can help identify missing IDs, unmatched records, or inconsistencies between datasets.

3.4 Practical workflow for joining data

In real-world projects, joining datasets often follows a consistent workflow:

1. Identify the key variable used to connect the datasets.

2. Inspect both datasets for duplicate keys or missing values.
3. Decide which join type is appropriate.
4. Perform the join.
5. Verify the number of rows before and after joining.
6. Inspect the final dataset for unexpected duplication or missing values.

Developing careful habits during joins is extremely important because join-related errors can propagate through the remainder of an analysis and affect final results.

3.5 Practice exercise

To reinforce the concepts introduced in this chapter, create two small datasets with a shared key variable and experiment with different join functions.

Try the following:

1. Perform a `left_join()` and compare the output with an `inner_join()`.
2. Add duplicate keys to one or both datasets and observe how the number of rows changes.
3. Use `anti_join()` to identify unmatched records.
4. Compare the dimensions of the datasets before and after joining.

As you work through these exercises, focus not only on obtaining the correct output, but also on understanding why the number of rows changes under different join relationships.

3.6 Chapter summary

In this chapter, you learned how datasets can be combined using joins in `dplyr`. You were introduced to join keys, common join relationships, and several important join functions including `left_join()`, `inner_join()`, and `anti_join()`.

You also learned why checking for duplicate keys is an essential part of reproducible data management workflows. Careful inspection of joins helps prevent data duplication, missing records, and inaccurate analytical results.

Chapter 4

Data Cleaning and Data Management

Data cleaning is one of the most important stages of any analytical workflow. In real-world projects, raw data are rarely ready for analysis immediately after import. Datasets often contain inconsistent variable names, missing values, formatting issues, duplicate records, encoding problems, or variables stored in unsuitable formats.

This chapter introduces several common data management tasks using the tidyverse. You will learn how to import multiple file types, inspect datasets, identify common data quality issues, reshape data into tidy formats, and recode variables into more useful forms.

Although the datasets used in this course are simplified for teaching purposes, the workflow closely reflects many real-world data cleaning tasks encountered in epidemiology, environmental health, and population data analysis.

4.1 Preparing the R environment

Before beginning any analysis, the required libraries should be loaded. For this chapter, we will primarily use packages from the tidyverse ecosystem, along with `ggplot2` and `readxl`.

If these packages are not already installed on your computer, they can be installed using:

```
install.packages(
  c("tidyverse", "ggplot2", "readxl")
)
```

Once installed, the libraries can be loaded into the R session.

```
library(tidyverse)
library(ggplot2)
library(readxl)
```

4.2 Exploring project files

Before importing data, it is often useful to inspect the contents of the project folders. The `list.files()` function allows you to display all files stored within a directory.

```
list.files("./Raw Data/")
```

```
## [1] "Corr_2016 copy.csv"            "Corr_2016.csv"
## [3] "Data Dictionary - Blank.xlsx"  "Data Dictionary - Filled.xlsx"
## [5] "deaths_2016.xlsx"              "Population_Estimates.csv"
## [7] "titanic.csv"                   "Titantic_DataDictionary.xlsx"
## [9] "Weather_data.csv"
```

You can also search for specific file types using the `pattern` argument.

```
list.files(
  "./Raw Data/",
  pattern = ".csv"
)
```

```
## [1] "Corr_2016 copy.csv"       "Corr_2016.csv"
## [3] "Population_Estimates.csv" "titanic.csv"
## [5] "Weather_data.csv"
```

```
list.files(
  "./Raw Data/",
  pattern = ".xlsx"
)
```

```
## [1] "Data Dictionary - Blank.xlsx"  "Data Dictionary - Filled.xlsx"
## [3] "deaths_2016.xlsx"              "Titantic_DataDictionary.xlsx"
```

Functions such as `list.files()` are frequently overlooked by beginners, but they become extremely useful when working with large projects that contain many datasets or outputs.

Some datasets may also be stored in compressed `.zip` files. These files should typically be extracted before analysis.

```
unzip(
  "./Raw Data/Corr_2016.zip",
  exdir = "./Raw Data"
)
```

The `exdir` argument specifies the folder where the extracted files should be saved. Since files only need to be unzipped once, this code is often commented out after the extraction is complete.

When working in R Markdown documents, avoid repeatedly running file extraction code unless necessary. Re-extracting files every time the document is knitted can slow down workflows and create duplicate files.

4.3 Description of the datasets

This course uses four datasets that simulate a simplified population health analysis workflow.

The mortality dataset contains records of deaths, including demographic information and ICD-10 cause-of-death codes. Although the dataset is based on realistic structures and counts, the data themselves are fictitious for teaching purposes.

The population dataset contains estimates of population counts by age group, sex, and Health Service Delivery Area in British Columbia.

The correspondence dataset links lower geographic units to larger health regions and service delivery areas. Correspondence files are commonly used in population health and environmental research because they allow different geographic datasets to be connected.

The environmental dataset contains weather-related variables by postal code, including measures related to temperature and cold exposure.

The overall analytical goal is to combine these datasets in order to calculate mortality rates and explore potential environmental relationships.

4.4 Importing data into R

Different file formats require different import functions. The `readr` package is commonly used for `.csv` files, while Excel files require functions from the `readxl` package.

The following code imports the datasets used in this chapter.

```
mort <- read_excel(
  "./Raw Data/deaths_2016.xlsx"
)

pop <- read_csv(
  "./Raw Data/Population_Estimates.csv"
)
```

```
## New names:
## Rows: 51 Columns: 25
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (2): Health Service Delivery Area, Gender dbl (23): ...1, Year, <1, 04-Jan,
## 09-May, 14-Oct, 15-19, 20-24, 25-29, 30-34...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```
corr <- read_csv(
  "./Raw Data/Corr_2016.csv",
  locale = readr::locale(
```

```
    encoding = "latin1"
  )
)
```

```
## Rows: 420963 Columns: 6
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): hrname_english, hrname_french
## dbl (4): dbuid2016, csduid2016, hruid2017, dbpop2016
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
env <- read_csv(
  "./Raw Data/Weather_data.csv"
)
```

```
## Rows: 116011 Columns: 8
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): POSTALCODE12
## dbl (7): WTHNRC12_01, WTHNRC12_02, WTHNRC12_03, WTHNRC12_04, WTHNRC12_05, WT...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Notice that the correspondence file uses a specific text encoding. Encoding problems are common when datasets contain accented characters or multilingual text.

4.5 Inspecting datasets

After importing data, it is important to examine the structure and contents of each dataset before beginning analysis.

Several functions provide useful high-level summaries.

```
str(mort)
```

```
## tibble [60,143 x 13] (S3: tbl_df/tbl/data.frame)
##  $ ID               : num [1:60143] 1e+06 1e+06 1e+06 1e+06 1e+06 ...
##  $ Sex              : chr [1:60143] "M" "F" "M" "M" ...
##  $ Cause_of_death   : chr [1:60143] "D47" "C34" "C26" "C16" ...
##  $ dbuid2016        : num [1:60143] 1.31e+10 4.81e+10 3.52e+10 2.47e+10 2.46e+10 ...
##  $ Location_of_death: num [1:60143] 5 3 6 1 6 5 4 1 3 2 ...
##  $ Marital_status   : num [1:60143] 1 1 2 1 1 1 1 1 1 2 ...
```

```
##  $ Postalcode       : chr [1:60143] "E1A1E9" "T2E0T2" "M4J1L1" "H9H1B4" ...
##  $ B_year           : num [1:60143] 1943 1943 1953 1923 1931 ...
##  $ B_month          : num [1:60143] 7 5 8 6 8 11 5 8 7 2 ...
##  $ B_day            : num [1:60143] 8 22 9 14 8 4 20 18 23 7 ...
##  $ D_year           : num [1:60143] 2016 2016 2016 2016 2016 ...
##  $ D_month          : num [1:60143] 9 12 4 11 4 2 2 6 5 1 ...
##  $ D_day            : num [1:60143] 22 14 21 21 4 6 9 13 25 16 ...
```

```
summary(mort)
```

```
##        ID              Sex            Cause_of_death       dbuid2016
##  Min.   :1000001   Length:60143       Length:60143       Min.   :1.001e+10
##  1st Qu.:1016187   Class :character   Class :character   1st Qu.:2.466e+10
##  Median :1034842   Mode  :character   Mode  :character   Median :3.521e+10
##  Mean   :1036699                                         Mean   :3.642e+10
##  3rd Qu.:1056221                                         3rd Qu.:4.715e+10
##  Max.   :1080708                                         Max.   :6.208e+10
##                                                          NA's   :4
##  Location_of_death Marital_status  Postalcode            B_year
##  Min.   :1.000     Min.   :1.0    Length:60143       Min.   :1827
##  1st Qu.:2.000     1st Qu.:1.0    Class :character   1st Qu.:1933
##  Median :4.000     Median :1.0    Mode  :character   Median :1941
##  Mean   :3.498     Mean   :1.5                       Mean   :1942
##  3rd Qu.:5.000     3rd Qu.:2.0                       3rd Qu.:1951
##  Max.   :6.000     Max.   :2.0                       Max.   :2016
##                                                      NA's   :24
##     B_month           B_day           D_year        D_month
##  Min.   : 1.000   Min.   : 1.00   Min.   :2016   Min.   : 1.000
##  1st Qu.: 4.000   1st Qu.: 8.00   1st Qu.:2016   1st Qu.: 4.000
##  Median : 7.000   Median :16.00   Median :2016   Median : 7.000
##  Mean   : 6.513   Mean   :15.74   Mean   :2016   Mean   : 6.521
##  3rd Qu.:10.000   3rd Qu.:23.00   3rd Qu.:2016   3rd Qu.: 9.000
##  Max.   :28.000   Max.   :31.00   Max.   :2016   Max.   :21.000
##  NA's   :26       NA's   :26
##      D_day
##  Min.   : 1.00
##  1st Qu.: 8.00
##  Median :16.00
##  Mean   :15.78
##  3rd Qu.:23.00
##  Max.   :31.00
##
```

```
str(pop)
```

```
## spc_tbl_ [51 x 25] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
```

```
##  $ ...1                        : num [1:51] 0 0 0 11 11 11 12 12 12 13 ...
##  $ Health Service Delivery Area: chr [1:51] "British Columbia" "British Columbia" "British C
##  $ Year                        : num [1:51] 2016 2016 2016 2016 2016 ...
##  $ Gender                      : chr [1:51] "M" "F" "T" "M" ...
##  $ <1                          : num [1:51] 22997 21760 44757 358 353 ...
##  $ 04-Jan                      : num [1:51] 93211 88300 181511 1507 1439 ...
##  $ 09-May                      : num [1:51] 121341 112777 234118 2058 2016 ...
##  $ 14-Oct                      : num [1:51] 119586 112424 232010 1838 1811 ...
##  $ 15-19                       : num [1:51] 140451 132226 272677 2179 2011 ...
##  $ 20-24                       : num [1:51] 170968 156161 327129 1935 1767 ...
##  $ 25-29                       : num [1:51] 159609 159389 318998 2055 2014 ...
##  $ 30-34                       : num [1:51] 162416 166826 329242 2332 2290 ...
##  $ 35-39                       : num [1:51] 155936 157628 313564 2527 2335 ...
##  $ 40-44                       : num [1:51] 151311 154187 305498 2399 2260 ...
##  $ 45-49                       : num [1:51] 161823 166603 328426 2465 2416 ...
##  $ 50-54                       : num [1:51] 172367 179295 351662 2882 2927 ...
##  $ 55-59                       : num [1:51] 172667 178878 351545 3337 3280 ...
##  $ 60-64                       : num [1:51] 156654 159977 316631 3166 3032 ...
##  $ 65-69                       : num [1:51] 139068 142469 281537 2961 2720 ...
##  $ 70-74                       : num [1:51] 99527 103484 203011 2035 1974 ...
##  $ 75-79                       : num [1:51] 69574 77160 146734 1428 1360 ...
##  $ 80-84                       : num [1:51] 49268 57639 106907 931 996 ...
##  $ 85-89                       : num [1:51] 28898 40789 69687 522 673 ...
##  $ 90+                         : num [1:51] 13366 28648 42014 242 437 ...
##  $ Total                       : num [1:51] 2361038 2396620 4757658 39157 38111 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   ...1 = col_double(),
##   ..   `Health Service Delivery Area` = col_character(),
##   ..   Year = col_double(),
##   ..   Gender = col_character(),
##   ..   `<1` = col_double(),
##   ..   `04-Jan` = col_double(),
##   ..   `09-May` = col_double(),
##   ..   `14-Oct` = col_double(),
##   ..   `15-19` = col_double(),
##   ..   `20-24` = col_double(),
##   ..   `25-29` = col_double(),
##   ..   `30-34` = col_double(),
##   ..   `35-39` = col_double(),
##   ..   `40-44` = col_double(),
##   ..   `45-49` = col_double(),
##   ..   `50-54` = col_double(),
##   ..   `55-59` = col_double(),
##   ..   `60-64` = col_double(),
##   ..   `65-69` = col_double(),
##   ..   `70-74` = col_double(),
##   ..   `75-79` = col_double(),
```

```
##   ..   `80-84` = col_double(),
##   ..   `85-89` = col_double(),
##   ..   `90+` = col_double(),
##   ..   Total = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

```
summary(pop)
```

```
##       ...1       Health Service Delivery Area      Year         Gender
##  Min.   : 0.00   Length:51                    Min.   :2016   Length:51
##  1st Qu.:14.00   Class :character             1st Qu.:2016   Class :character
##  Median :31.00   Mode  :character             Median :2016   Mode  :character
##  Mean   :29.06                                Mean   :2016
##  3rd Qu.:42.00                                3rd Qu.:2016
##  Max.   :53.00                                Max.   :2016
##        <1            04-Jan           09-May           14-Oct
##  Min.   :  276   Min.   :  1356   Min.   :  1755   Min.   :  1811
##  1st Qu.:  714   1st Qu.:  3118   1st Qu.:  4194   1st Qu.:  3910
##  Median : 1425   Median :  5663   Median :  8002   Median :  7808
##  Mean   : 3510   Mean   : 14236   Mean   : 18362   Mean   : 18197
##  3rd Qu.: 2802   3rd Qu.: 11025   3rd Qu.: 15384   3rd Qu.: 15350
##  Max.   :44757   Max.   :181511   Max.   :234118   Max.   :232010
##      15-19            20-24            25-29            30-34
##  Min.   :  1919   Min.   :  1577   Min.   :  1640   Min.   :  2069
##  1st Qu.:  4240   1st Qu.:  4214   1st Qu.:  4169   1st Qu.:  4415
##  Median :  8836   Median :  9193   Median :  8646   Median :  8830
##  Mean   : 21386   Mean   : 25657   Mean   : 25019   Mean   : 25823
##  3rd Qu.: 17866   3rd Qu.: 22940   3rd Qu.: 21291   3rd Qu.: 22644
##  Max.   :272677   Max.   :327129   Max.   :318998   Max.   :329242
##      35-39            40-44            45-49            50-54
##  Min.   :  2092   Min.   :  2008   Min.   :  2040   Min.   :  2137
##  1st Qu.:  4572   1st Qu.:  4483   1st Qu.:  4859   1st Qu.:  5431
##  Median :  9490   Median :  9551   Median :  9930   Median : 10666
##  Mean   : 24593   Mean   : 23961   Mean   : 25759   Mean   : 27581
##  3rd Qu.: 21896   3rd Qu.: 21345   3rd Qu.: 23211   3rd Qu.: 24455
##  Max.   :313564   Max.   :305498   Max.   :328426   Max.   :351662
##      55-59            60-64            65-69            70-74
##  Min.   :  2038   Min.   :  1550   Min.   :  1119   Min.   :   784
##  1st Qu.:  5684   1st Qu.:  5349   1st Qu.:  5006   1st Qu.:  3628
##  Median : 10972   Median : 10698   Median : 10011   Median :  7256
##  Mean   : 27572   Mean   : 24834   Mean   : 22081   Mean   : 15922
##  3rd Qu.: 22498   3rd Qu.: 19486   3rd Qu.: 16891   3rd Qu.: 12128
##  Max.   :351545   Max.   :316631   Max.   :281537   Max.   :203011
##      75-79            80-84            85-89            90+
##  Min.   :   538   Min.   :   351   Min.   :  196   Min.   :  114.0
##  1st Qu.:  2332   1st Qu.:  1612   1st Qu.:  973   1st Qu.:  536.5
```

```
##  Median :  5026   Median :  3867   Median : 2636   Median : 1394.0
##  Mean   : 11509   Mean   :  8385   Mean   : 5466   Mean   : 3295.2
##  3rd Qu.:  9268   3rd Qu.:  6582   3rd Qu.: 4543   3rd Qu.: 2955.0
##  Max.   :146734   Max.   :106907   Max.   :69687   Max.   :42014.0
##      Total
##  Min.   :  32171
##  1st Qu.:  70681
##  Median : 145321
##  Mean   : 373150
##  3rd Qu.: 329569
##  Max.   :4757658
```

```
str(corr)
```

```
## spc_tbl_ [420,963 x 6] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ dbuid2016     : num [1:420963] 1e+10 1e+10 1e+10 1e+10 1e+10 ...
##  $ csduid2016    : num [1:420963] 1e+06 1e+06 1e+06 1e+06 1e+06 ...
##  $ hruid2017     : num [1:420963] 1011 1011 1011 1011 1011 ...
##  $ hrname_english: chr [1:420963] "Eastern Regional Health Authority" "Eastern Regional Heal
##  $ hrname_french : chr [1:420963] "Eastern Regional Health Authority" "Eastern Regional Heal
##  $ dbpop2016     : num [1:420963] 160 25 268 53 71 217 39 120 154 111 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   dbuid2016 = col_double(),
##   ..   csduid2016 = col_double(),
##   ..   hruid2017 = col_double(),
##   ..   hrname_english = col_character(),
##   ..   hrname_french = col_character(),
##   ..   dbpop2016 = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

```
summary(corr)
```

```
##    dbuid2016           csduid2016        hruid2017    hrname_english
##  Min.   :1.001e+10   Min.   :1001101   Min.   :1011   Length:420963
##  1st Qu.:2.466e+10   1st Qu.:2466023   1st Qu.:2414   Class :character
##  Median :3.558e+10   Median :3558090   Median :3595   Mode  :character
##  Mean   :3.856e+10   Mean   :3855572   Mean   :3856
##  3rd Qu.:4.806e+10   3rd Qu.:4806016   3rd Qu.:4832
##  Max.   :6.208e+10   Max.   :6208098   Max.   :6201
##
##  hrname_french        dbpop2016
##  Length:420963      Min.   :   0.00
##  Class :character   1st Qu.:   5.00
##  Mode  :character   Median :  26.00
```

```
##                     Mean   :  69.44
##                     3rd Qu.:  77.00
##                     Max.   :7607.00
##                     NA's   :35
```

```
str(env)
```

```
## spc_tbl_ [116,011 x 8] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ POSTALCODE12: chr [1:116011] "V0C1E0" "V0C1W0" "V0C2X0" "V0C2Z0" ...
##  $ WTHNRC12_01 : num [1:116011] 27.1 30.3 28.2 28.7 23.1 ...
##  $ WTHNRC12_02 : num [1:116011] -46.7 -46 -42.3 -44.9 -35 ...
##  $ WTHNRC12_03 : num [1:116011] -2.605 -1.868 -1.963 -1.286 0.338 ...
##  $ WTHNRC12_04 : num [1:116011] 2.91 3.92 4.01 4.32 4.33 ...
##  $ WTHNRC12_05 : num [1:116011] -8.12 -7.65 -7.93 -6.89 -3.65 ...
##  $ WTHNRC12_06 : num [1:116011] 11.04 11.57 11.94 11.2 7.98 ...
##  $ WTHNRC12_07 : num [1:116011] 171 198 248 175 310 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   POSTALCODE12 = col_character(),
##   ..   WTHNRC12_01 = col_double(),
##   ..   WTHNRC12_02 = col_double(),
##   ..   WTHNRC12_03 = col_double(),
##   ..   WTHNRC12_04 = col_double(),
##   ..   WTHNRC12_05 = col_double(),
##   ..   WTHNRC12_06 = col_double(),
##   ..   WTHNRC12_07 = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

```
summary(env)
```

```
##  POSTALCODE12        WTHNRC12_01     WTHNRC12_02      WTHNRC12_03
##  Length:116011      Min.   :19.28   Min.   :-46.74   Min.   :-2.605
##  Class :character   1st Qu.:31.07   1st Qu.:-11.44   1st Qu.: 9.693
##  Mode  :character   Median :31.28   Median : -8.40   Median :10.140
##                     Mean   :31.62   Mean   :-11.67   Mean   : 9.573
##                     3rd Qu.:31.76   3rd Qu.: -7.49   3rd Qu.:10.449
##                     Max.   :39.07   Max.   : -4.48   Max.   :10.551
##   WTHNRC12_04      WTHNRC12_05      WTHNRC12_06      WTHNRC12_07
##  Min.   : 2.913   Min.   :-8.122   Min.   : 4.336   Min.   : 104.6
##  1st Qu.:13.454   1st Qu.: 5.750   1st Qu.: 6.936   1st Qu.:1072.1
##  Median :13.887   Median : 6.612   Median : 7.151   Median :1316.4
##  Mean   :13.521   Mean   : 5.625   Mean   : 7.896   Mean   :1110.7
##  3rd Qu.:14.006   3rd Qu.: 6.893   3rd Qu.: 7.935   3rd Qu.:1399.0
##  Max.   :16.227   Max.   : 7.132   Max.   :13.498   Max.   :3076.5
```

The `str()` function displays the structure of the dataset, including variable types and dimensions. The `summary()` function provides summary statistics and quick descriptive information.

Another useful function is `glimpse()` from `dplyr`.

```
glimpse(mort)
```

```
## Rows: 60,143
## Columns: 13
## $ ID                <dbl> 1000001, 1000002, 1000003, 1000004, 1000005, 1000006~
## $ Sex               <chr> "M", "F", "M", "M", "M", "F", "F", "F", "F", "M", "F~
## $ Cause_of_death    <chr> "D47", "C34", "C26", "C16", "C22", "C50", "C85", "C5~
## $ dbuid2016         <dbl> 13070175019, 48060924001, 35204502002, 24663048013, ~
## $ Location_of_death <dbl> 5, 3, 6, 1, 6, 5, 4, 1, 3, 2, 5, 1, 3, 4, 1, 5, 2, 6~
## $ Marital_status    <dbl> 1, 1, 2, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 1, 2~
## $ Postalcode        <chr> "E1A1E9", "T2E0T2", "M4J1L1", "H9H1B4", "J2W2B6", "L~
## $ B_year            <dbl> 1943, 1943, 1953, 1923, 1931, 1933, 1918, 1972, 1922~
## $ B_month           <dbl> 7, 5, 8, 6, 8, 11, 5, 8, 7, 2, 11, 3, 5, 9, 8, 12, 5~
## $ B_day             <dbl> 8, 22, 9, 14, 8, 4, 20, 18, 23, 7, 8, 6, 1, 8, 14, 3~
## $ D_year            <dbl> 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016~
## $ D_month           <dbl> 9, 12, 4, 11, 4, 2, 2, 6, 5, 1, 2, 10, 6, 2, 7, 8, 6~
## $ D_day             <dbl> 22, 14, 21, 21, 4, 6, 9, 13, 25, 16, 22, 17, 15, 7, ~
```

Compared with `str()`, `glimpse()` often provides a cleaner and easier-to-read overview of the data frame.

Initial data inspection is one of the most important parts of data cleaning because it helps identify:

- missing values;
- unusual variable names;
- incorrect data types;
- inconsistent formatting; and
- unexpected values.

4.6 Exploring and validating the data

A useful next step is performing simple exploratory checks to understand the datasets more fully.

For example, we can identify all health regions in the correspondence dataset.

```
corr %>%
  distinct(hrname_english) %>%
  mutate(
    hrname_english = iconv(
      hrname_english,
      from = "latin1",
      to = "UTF-8",
```

```
      sub = ""
    )
  ) %>%
  arrange(hrname_english)
```

```
## # A tibble: 96 x 1
##    hrname_english
##    <chr>
##  1 Athabasca Health Authority
##  2 Brant County Health Unit
##  3 Calgary Zone
##  4 Central Regional Health Authority
##  5 Central Vancouver Island Health Service Delivery Area
##  6 Central Zone
##  7 Chatham-Kent Health Unit
##  8 City of Hamilton Health Unit
##  9 City of Toronto Health Unit
## 10 Cypress Regional Health Authority
## # i 86 more rows
```

When working with multilingual datasets, encoding issues may cause characters to display incorrectly. Re-importing the file using the appropriate encoding often resolves these problems.

```
corr <- read_csv(
  "./Raw Data/Corr_2016.csv",
  locale = readr::locale(
    encoding = "latin1"
  )
)
```

```
## Rows: 420963 Columns: 6
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): hrname_english, hrname_french
## dbl (4): dbuid2016, csduid2016, hruid2017, dbpop2016
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
corr %>%
  distinct(hrname_english) %>%
  arrange(hrname_english)
```

```
## # A tibble: 96 x 1
##    hrname_english
```

```
##    <chr>
##  1 Athabasca Health Authority
##  2 Brant County Health Unit
##  3 Calgary Zone
##  4 Central Regional Health Authority
##  5 Central Vancouver Island Health Service Delivery Area
##  6 Central Zone
##  7 Chatham-Kent Health Unit
##  8 City of Hamilton Health Unit
##  9 City of Toronto Health Unit
## 10 Cypress Regional Health Authority
## # i 86 more rows
```

We can also calculate the number of health regions in the dataset.

```
no_regions <- corr %>%
  distinct(hrname_english) %>%
  nrow()
```

There are 96 health regions in the correspondence dataset.

Additional exploratory summaries can help validate the imported data.

```
mort %>%
  count(Sex)
```

```
## # A tibble: 3 x 2
##   Sex       n
##   <chr> <int>
## 1 F     28630
## 2 M     31500
## 3 <NA>     13
```

```
pop %>%
  summarize(
    Total_below_1 = sum(`<1`)
  )
```

```
## # A tibble: 1 x 1
##   Total_below_1
##           <dbl>
## 1        179028
```

These simple checks help verify that the datasets were imported correctly and that the variables contain plausible values.

4.7 Cleaning variable names and reshaping data

One common issue in real-world datasets is poorly formatted variable names. Variables that begin with numbers or contain symbols can make coding more difficult.

For example:

```
names(pop)
```

```
##  [1] "...1"                          "Health Service Delivery Area"
##  [3] "Year"                          "Gender"
##  [5] "<1"                            "04-Jan"
##  [7] "09-May"                        "14-Oct"
##  [9] "15-19"                         "20-24"
## [11] "25-29"                         "30-34"
## [13] "35-39"                         "40-44"
## [15] "45-49"                         "50-54"
## [17] "55-59"                         "60-64"
## [19] "65-69"                         "70-74"
## [21] "75-79"                         "80-84"
## [23] "85-89"                         "90+"
## [25] "Total"
```

```
colnames(pop)
```

```
##  [1] "...1"                          "Health Service Delivery Area"
##  [3] "Year"                          "Gender"
##  [5] "<1"                            "04-Jan"
##  [7] "09-May"                        "14-Oct"
##  [9] "15-19"                         "20-24"
## [11] "25-29"                         "30-34"
## [13] "35-39"                         "40-44"
## [15] "45-49"                         "50-54"
## [17] "55-59"                         "60-64"
## [19] "65-69"                         "70-74"
## [21] "75-79"                         "80-84"
## [23] "85-89"                         "90+"
## [25] "Total"
```

Some age-group variables contain names such as `"04-Jan"` or `"09-May"`, which are not ideal variable names for analysis.

One approach is to rename problematic variables directly.

```
pop2 <- pop %>%
  rename(
    "01-04" = "04-Jan",
```

```
    "05-09" = "09-May",
    "10-14" = "14-Oct"
  )
```

Another important data-cleaning task is reshaping data into a *tidy* format. In tidy data, each variable has its own column, each observation has its own row, and each value occupies a single cell.

The `pivot_longer()` function is commonly used to convert wide datasets into long format.

```
pop_long <- pop %>%

  select(
    -any_of(c("X1", "Total"))
  ) %>%

  pivot_longer(
    cols = -any_of(
      c(
        "Year",
        "Gender",
        "Health Service Delivery Area"
      )
    ),
    names_to = "Age",
    values_to = "Value"
  )

pop_long
```

```
## # A tibble: 1,071 x 5
##    `Health Service Delivery Area`  Year Gender Age     Value
##    <chr>                          <dbl> <chr>  <chr>   <dbl>
##  1 British Columbia                2016 M      ...1        0
##  2 British Columbia                2016 M      <1      22997
##  3 British Columbia                2016 M      04-Jan  93211
##  4 British Columbia                2016 M      09-May 121341
##  5 British Columbia                2016 M      14-Oct 119586
##  6 British Columbia                2016 M      15-19  140451
##  7 British Columbia                2016 M      20-24  170968
##  8 British Columbia                2016 M      25-29  159609
##  9 British Columbia                2016 M      30-34  162416
## 10 British Columbia                2016 M      35-39  155936
## # i 1,061 more rows
```

The resulting dataset stores age groups within a single variable rather than across many separate columns.

After reshaping the data, some values may still require cleaning or recoding. The `case_when()` function is often used for this purpose.

```
pop_long <- pop_long %>%

  mutate(
    Age = case_when(
      Age == "04-Jan" ~ "01-04",
      Age == "09-May" ~ "05-09",
      Age == "14-Oct" ~ "10-14",
      TRUE ~ Age
    )
  )
```

The final line, `TRUE ~ Age`, is important because it preserves all values that do not match the earlier conditions.

If this line is removed, unmatched rows become missing values.

```
pop %>%

  select(
    -any_of(c("X1", "Total"))
  ) %>%

  pivot_longer(
    cols = -any_of(
      c(
        "Year",
        "Gender",
        "Health Service Delivery Area"
      )
    ),
    names_to = "Age",
    values_to = "Value"
  ) %>%

  mutate(
    Age = case_when(
      Age == "04-Jan" ~ "01-04",
      Age == "09-May" ~ "05-09",
      Age == "14-Oct" ~ "10-14"
    )
  )
```

```
## # A tibble: 1,071 x 5
##    `Health Service Delivery Area`  Year Gender Age    Value
##    <chr>                          <dbl> <chr>  <chr>  <dbl>
```

```
##  1 British Columbia                2016 M      <NA>       0
##  2 British Columbia                2016 M      <NA>   22997
##  3 British Columbia                2016 M      01-04  93211
##  4 British Columbia                2016 M      05-09 121341
##  5 British Columbia                2016 M      10-14 119586
##  6 British Columbia                2016 M      <NA>  140451
##  7 British Columbia                2016 M      <NA>  170968
##  8 British Columbia                2016 M      <NA>  159609
##  9 British Columbia                2016 M      <NA>  162416
## 10 British Columbia                2016 M      <NA>  155936
## # i 1,061 more rows
```

Always verify the output after recoding variables. Small coding mistakes can unintentionally convert valid values into missing values.

4.8 Practical considerations for R Markdown workflows

When working with R Markdown documents, remember that every code chunk is executed again during knitting. Code that modifies files, unzips folders, or downloads data should therefore be used carefully.

As projects grow larger, keeping code organized and avoiding repeated operations becomes increasingly important for reproducible workflows.

You should also begin thinking about how analyses are documented. A good R Markdown document not only produces correct outputs, but also explains the reasoning behind each analytical step.

4.9 Chapter summary

In this chapter, you learned several important data cleaning and management techniques using the tidyverse. You imported multiple datasets, explored project files, inspected dataset structures, handled encoding issues, cleaned variable names, reshaped data into tidy formats, and recoded variables using `case_when()`.

These skills form the foundation of reproducible data analysis workflows and will be used extensively throughout the remainder of the course.

Chapter 5

Strings and Regular Expressions

Character variables are extremely common in real-world datasets. Names, addresses, postal codes, diagnostic codes, survey responses, and file names are all examples of text-based data that often require cleaning before analysis can begin.

This chapter introduces tools for working with strings and regular expressions in R using the tidyverse and the `stringr` package. You will learn how to separate text variables into multiple columns, remove unnecessary spaces, extract patterns from text, reshape complex datasets, and use regular expressions to identify meaningful information hidden inside character strings.

In addition to text cleaning, this chapter also introduces important data reshaping techniques using `pivot_longer()`, which is commonly used to convert messy wide-format datasets into tidy long-format structures suitable for analysis.

5.1 Working with character data in R

The tidyverse includes several useful tools for working with text data. In this chapter, we will primarily use functions from `stringr`, which provides a consistent and readable framework for string manipulation.

Begin by loading the required libraries.

```
library(tidyverse)
library(stringr)
```

Throughout this chapter, we will work with two example datasets:

- the Titanic dataset, which will be used to practice cleaning names and extracting passenger titles; and
- the WHO tuberculosis dataset, which will be used to demonstrate reshaping and decoding complex variable names.

5.2 Cleaning and separating character variables

The Titanic dataset contains a variable called `Name`, which stores multiple pieces of information within a single text field.

We begin by importing the data and converting selected variables into factors.

```
titanic <- read_csv(
  "./Raw Data/titanic.csv",
  col_types = cols(
    Survived = col_factor(),
    Sex = col_factor()
  )
)
```

Before working with the names, we perform a small amount of initial cleaning. Records with missing survival information are removed, and a new variable called `family_size` is created.

```
titanic <- titanic %>%

  filter(!is.na(Survived)) %>%

  mutate(
    family_size = SibSp + Parch + 1
  )

head(titanic)
```

```
## # A tibble: 6 x 13
##   PassengerId Survived Pclass Name    Sex     Age SibSp Parch Ticket  Fare Cabin
##         <dbl> <fct>     <dbl> <chr>   <fct> <dbl> <dbl> <dbl> <chr>  <dbl> <chr>
## 1           1 0             3 Braund~ male     22     1     0 A/5 2~  7.25 <NA>
## 2           2 1             1 Cuming~ fema~    38     1     0 PC 17~ 71.3  C85
## 3           3 1             3 Heikki~ fema~    26     0     0 STON/~  7.92 <NA>
## 4           4 1             1 Futrel~ fema~    35     1     0 113803 53.1  C123
## 5           5 0             3 Allen,~ male     35     0     0 373450  8.05 <NA>
## 6           6 0             3 Moran,~ male     NA     0     0 330877  8.46 <NA>
## # i 2 more variables: Embarked <chr>, family_size <dbl>
```

The `Name` variable contains both last names and first names in a single column. To make the data easier to analyze, we can separate this variable into two new columns.

```
titanic <- titanic %>%

  separate(
    col = Name,
    into = c("Last_name", "First_name"),
```

```
    sep = ","
  )

head(
  titanic %>%
    select(Last_name, First_name)
)
```

```
## # A tibble: 6 x 2
##   Last_name First_name
##   <chr>     <chr>
## 1 Braund    " Mr. Owen Harris"
## 2 Cumings   " Mrs. John Bradley (Florence Briggs Thayer)"
## 3 Heikkinen " Miss. Laina"
## 4 Futrelle  " Mrs. Jacques Heath (Lily May Peel)"
## 5 Allen     " Mr. William Henry"
## 6 Moran     " Mr. James"
```

After separating the names, some records contain extra spaces at the beginning of the `First_name` variable. These spaces can be removed using `str_trim()`.

```
titanic <- titanic %>%

  mutate(
    First_name = str_trim(
      First_name,
      side = "left"
    )
  )

head(
  titanic %>%
    select(Last_name, First_name)
)
```

```
## # A tibble: 6 x 2
##   Last_name First_name
##   <chr>     <chr>
## 1 Braund    Mr. Owen Harris
## 2 Cumings   Mrs. John Bradley (Florence Briggs Thayer)
## 3 Heikkinen Miss. Laina
## 4 Futrelle  Mrs. Jacques Heath (Lily May Peel)
## 5 Allen     Mr. William Henry
## 6 Moran     Mr. James
```

Cleaning whitespace is a small but important part of text processing because extra spaces can interfere with matching, grouping, and filtering operations.

5.3 Extracting patterns with regular expressions

Many text variables contain structured patterns that can be extracted using regular expressions, often called *regex*.

In the Titanic dataset, passenger titles such as `"Mr"`, `"Mrs"`, `"Miss"`, and `"Master"` appear within the `First_name` variable. We can extract these titles using the `str_extract()` function.

```
titanic <- titanic %>%

  mutate(
    Title = str_extract(
      First_name,
      "^[^.]+"
    )
  )

head(
  titanic %>%
    select(First_name, Title)
)
```

```
## # A tibble: 6 x 2
##   First_name                                  Title
##   <chr>                                       <chr>
## 1 Mr. Owen Harris                             Mr
## 2 Mrs. John Bradley (Florence Briggs Thayer) Mrs
## 3 Miss. Laina                                 Miss
## 4 Mrs. Jacques Heath (Lily May Peel)          Mrs
## 5 Mr. William Henry                           Mr
## 6 Mr. James                                   Mr
```

The regular expression `^[^.]+` can be interpreted as follows:

- `^` indicates the beginning of the string;
- `[^.]` means any character except a period; and
- `+` means one or more occurrences of the previous pattern.

Together, the expression extracts all characters from the beginning of the string until the first period is reached.

Regular expressions can initially appear intimidating, but they become extremely powerful once you begin recognizing common text patterns.

When learning regular expressions, it is often helpful to interpret them one symbol at a time rather than trying to understand the entire pattern at once.

5.4 Creating reusable functions

When the same sequence of operations must be repeated multiple times, it is often useful to place the code inside a function.

The following function trims whitespace and extracts passenger titles.

```
create_title <- function(firstname) {

  firstname %>%

    str_trim(side = "left") %>%

    str_extract("^[^.]+")
}
```

We can then apply the function directly to the dataset.

```
titanic <- titanic %>%

  mutate(
    Title_from_function =
      create_title(First_name)
  )

head(
  titanic %>%
    select(
      First_name,
      Title,
      Title_from_function
    )
)
```

```
## # A tibble: 6 x 3
##   First_name                                  Title Title_from_function
##   <chr>                                       <chr> <chr>
## 1 Mr. Owen Harris                             Mr    Mr
## 2 Mrs. John Bradley (Florence Briggs Thayer) Mrs   Mrs
## 3 Miss. Laina                                 Miss  Miss
## 4 Mrs. Jacques Heath (Lily May Peel)          Mrs   Mrs
## 5 Mr. William Henry                           Mr    Mr
## 6 Mr. James                                   Mr    Mr
```

Creating functions improves readability and reduces repeated code, especially in large projects.

5.5 Summarizing and visualizing categorical data

Once titles have been extracted, we can summarize the number of passengers within each title group.

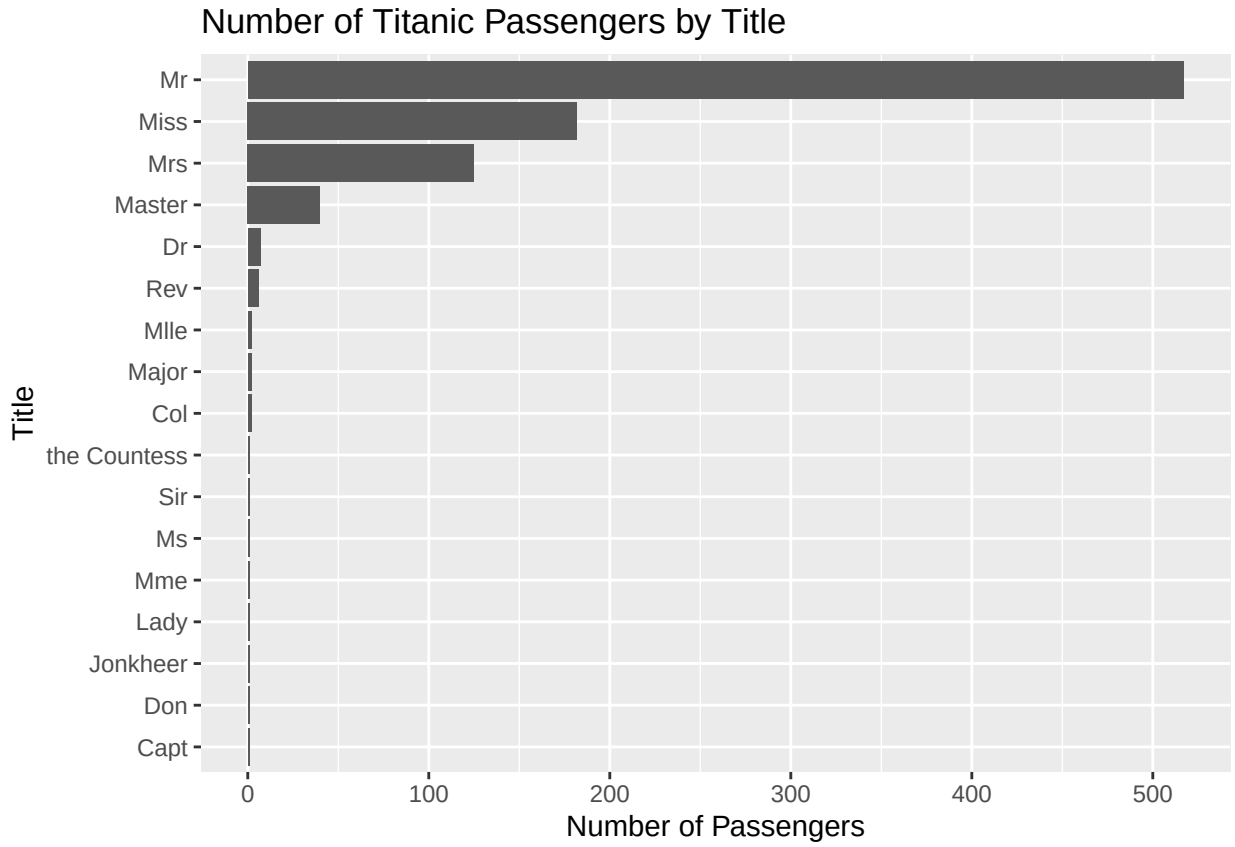
```
title_summary <- titanic %>%

  count(Title, sort = TRUE)

title_summary
```

```
## # A tibble: 17 x 2
##    Title            n
##    <chr>        <int>
##  1 Mr             517
##  2 Miss           182
##  3 Mrs            125
##  4 Master          40
##  5 Dr               7
##  6 Rev              6
##  7 Col              2
##  8 Major            2
##  9 Mlle             2
## 10 Capt             1
## 11 Don              1
## 12 Jonkheer         1
## 13 Lady             1
## 14 Mme              1
## 15 Ms               1
## 16 Sir              1
## 17 the Countess     1
```

We can also visualize the distribution using `ggplot2`.

```
title_summary %>%

  ggplot(
    aes(
      x = reorder(Title, n),
      y = n
    )
  ) +

  geom_col() +

  coord_flip() +
```

```
  labs(
    title = "Number of Titanic Passengers by Title",
    x = "Title",
    y = "Number of Passengers"
  )
```



Visual summaries are often useful for identifying unusual categories, inconsistencies, or unexpected values within character variables.

5.6 Reshaping complex datasets

The WHO tuberculosis dataset provides an excellent example of why data reshaping is often necessary.

```
data(who)

glimpse(who)
```

```
## Rows: 7,240
## Columns: 60
## $ country      <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan~
```

```
## $ iso2         <chr> "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF~
## $ iso3         <chr> "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "~
## $ year         <dbl> 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 198~
## $ new_sp_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sp_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_sn_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ new_ep_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

```
## $ newrel_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ newrel_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

Many column names in this dataset are not true variables. Instead, the column names themselves contain several pieces of information.

For example:

```
new_sp_m014
```

contains:

- the diagnosis type;
- the sex category; and
- the age group.

This structure violates the principles of tidy data because multiple variables are embedded inside the column names.

To clean the dataset, we first reshape it from wide format into long format using `pivot_longer()`.

```
who1 <- who %>%

  pivot_longer(
    cols = new_sp_m014:newrel_f65,
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  )

who1
```

```
## # A tibble: 76,046 x 6
##    country     iso2  iso3   year key          cases
##    <chr>       <chr> <chr> <dbl> <chr>        <dbl>
##  1 Afghanistan AF    AFG    1997 new_sp_m014      0
##  2 Afghanistan AF    AFG    1997 new_sp_m1524    10
##  3 Afghanistan AF    AFG    1997 new_sp_m2534     6
```

```
##  4 Afghanistan AF    AFG    1997 new_sp_m3544     3
##  5 Afghanistan AF    AFG    1997 new_sp_m4554     5
##  6 Afghanistan AF    AFG    1997 new_sp_m5564     2
##  7 Afghanistan AF    AFG    1997 new_sp_m65       0
##  8 Afghanistan AF    AFG    1997 new_sp_f014      5
##  9 Afghanistan AF    AFG    1997 new_sp_f1524    38
## 10 Afghanistan AF    AFG    1997 new_sp_f2534    36
## # i 76,036 more rows
```

After reshaping the data, we correct inconsistent variable naming.

```
who2 <- who1 %>%

  mutate(
    key = str_replace(
      key,
      "newrel",
      "new_rel"
    )
  )

who2
```

```
## # A tibble: 76,046 x 6
##    country     iso2  iso3   year key          cases
##    <chr>       <chr> <chr> <dbl> <chr>        <dbl>
##  1 Afghanistan AF    AFG    1997 new_sp_m014      0
##  2 Afghanistan AF    AFG    1997 new_sp_m1524    10
##  3 Afghanistan AF    AFG    1997 new_sp_m2534     6
##  4 Afghanistan AF    AFG    1997 new_sp_m3544     3
##  5 Afghanistan AF    AFG    1997 new_sp_m4554     5
##  6 Afghanistan AF    AFG    1997 new_sp_m5564     2
##  7 Afghanistan AF    AFG    1997 new_sp_m65       0
##  8 Afghanistan AF    AFG    1997 new_sp_f014      5
##  9 Afghanistan AF    AFG    1997 new_sp_f1524    38
## 10 Afghanistan AF    AFG    1997 new_sp_f2534    36
## # i 76,036 more rows
```

The combined `key` variable can then be separated into multiple meaningful variables.

```
who3 <- who2 %>%

  separate(
    col = key,
    into = c(
      "new",
      "type",
```

```
      "sexage"
    ),
    sep = "_"
  )

who3
```

```
## # A tibble: 76,046 x 8
##    country     iso2  iso3   year new   type  sexage cases
##    <chr>       <chr> <chr> <dbl> <chr> <chr> <chr>  <dbl>
##  1 Afghanistan AF    AFG    1997 new   sp    m014       0
##  2 Afghanistan AF    AFG    1997 new   sp    m1524     10
##  3 Afghanistan AF    AFG    1997 new   sp    m2534      6
##  4 Afghanistan AF    AFG    1997 new   sp    m3544      3
##  5 Afghanistan AF    AFG    1997 new   sp    m4554      5
##  6 Afghanistan AF    AFG    1997 new   sp    m5564      2
##  7 Afghanistan AF    AFG    1997 new   sp    m65        0
##  8 Afghanistan AF    AFG    1997 new   sp    f014       5
##  9 Afghanistan AF    AFG    1997 new   sp    f1524     38
## 10 Afghanistan AF    AFG    1997 new   sp    f2534     36
## # i 76,036 more rows
```

Finally, the `sexage` variable can be divided into separate sex and age variables.

```
who4 <- who3 %>%

  separate(
    col = sexage,
    into = c("sex", "age"),
    sep = 1
  )

who4
```

```
## # A tibble: 76,046 x 9
##    country     iso2  iso3   year new   type  sex   age   cases
##    <chr>       <chr> <chr> <dbl> <chr> <chr> <chr> <chr> <dbl>
##  1 Afghanistan AF    AFG    1997 new   sp    m     014       0
##  2 Afghanistan AF    AFG    1997 new   sp    m     1524     10
##  3 Afghanistan AF    AFG    1997 new   sp    m     2534      6
##  4 Afghanistan AF    AFG    1997 new   sp    m     3544      3
##  5 Afghanistan AF    AFG    1997 new   sp    m     4554      5
##  6 Afghanistan AF    AFG    1997 new   sp    m     5564      2
##  7 Afghanistan AF    AFG    1997 new   sp    m     65        0
##  8 Afghanistan AF    AFG    1997 new   sp    f     014       5
##  9 Afghanistan AF    AFG    1997 new   sp    f     1524     38
```

```
## 10 Afghanistan AF    AFG    1997 new    sp    f      2534      36
## # i 76,036 more rows
```

The entire cleaning workflow can also be written as one continuous pipeline.

```
who_clean <- who %>%

  pivot_longer(
    cols = new_sp_m014:newrel_f65,
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%

  mutate(
    key = str_replace(
      key,
      "newrel",
      "new_rel"
    )
  ) %>%

  separate(
    key,
    into = c(
      "new",
      "type",
      "sexage"
    ),
    sep = "_"
  ) %>%

  select(
    -new,
    -iso2,
    -iso3
  ) %>%

  separate(
    sexage,
    into = c("sex", "age"),
    sep = 1
  )

who_clean
```

```
## # A tibble: 76,046 x 6
```

```
##    country      year type  sex   age   cases
##    <chr>       <dbl> <chr> <chr> <chr> <dbl>
##  1 Afghanistan  1997 sp    m     014       0
##  2 Afghanistan  1997 sp    m     1524     10
##  3 Afghanistan  1997 sp    m     2534      6
##  4 Afghanistan  1997 sp    m     3544      3
##  5 Afghanistan  1997 sp    m     4554      5
##  6 Afghanistan  1997 sp    m     5564      2
##  7 Afghanistan  1997 sp    m     65        0
##  8 Afghanistan  1997 sp    f     014       5
##  9 Afghanistan  1997 sp    f     1524     38
## 10 Afghanistan  1997 sp    f     2534     36
## # i 76,036 more rows
```

5.7 Using regular expressions inside pivot_longer()

An alternative approach uses regular expressions directly inside `pivot_longer()` to extract multiple variables at once.

```
who_longer <- who %>%

  pivot_longer(
    cols = new_sp_m014:newrel_f65,

    names_to = c(
      "diagnosis",
      "gender",
      "age"
    ),

    names_pattern =
      "new_?(.*)_(.)(.*)",

    values_to = "count",

    values_drop_na = TRUE
  )

who_longer
```

```
## # A tibble: 76,046 x 8
##    country     iso2  iso3   year diagnosis gender age   count
##    <chr>       <chr> <chr> <dbl> <chr>     <chr>  <chr> <dbl>
##  1 Afghanistan AF    AFG    1997 sp        m      014       0
##  2 Afghanistan AF    AFG    1997 sp        m      1524     10
##  3 Afghanistan AF    AFG    1997 sp        m      2534      6
```

```
##  4 Afghanistan AF    AFG    1997 sp        m      3544      3
##  5 Afghanistan AF    AFG    1997 sp        m      4554      5
##  6 Afghanistan AF    AFG    1997 sp        m      5564      2
##  7 Afghanistan AF    AFG    1997 sp        m      65        0
##  8 Afghanistan AF    AFG    1997 sp        f      014       5
##  9 Afghanistan AF    AFG    1997 sp        f      1524     38
## 10 Afghanistan AF    AFG    1997 sp        f      2534     36
## # i 76,036 more rows
```

The regular expression used here contains several capture groups:

- `(.*)` extracts the diagnosis type;
- `(.)` extracts one character representing sex; and
- the final `(.*)` extracts the age group.

These extracted components are automatically assigned to the variables listed in `names_to`.

This approach demonstrates how regular expressions can be integrated directly into data reshaping workflows.

5.8 Missing values and interpretation

During reshaping, we used the argument:

```
values_drop_na = TRUE
```

This removes rows containing missing values.

However, it is important to remember that missing values are not always equivalent to zero values.

An `NA` typically means that the value is unknown or unavailable, while a value of `0` indicates that the quantity is known and equal to zero.

Understanding the meaning of missing values is an essential part of responsible data analysis.

Never assume that missing values and zeros represent the same thing. Incorrect handling of missing data can substantially alter analytical results.

5.9 Additional practice

The cleaned datasets created in this chapter can now be used for additional exploratory analysis.

Possible exercises include:

- identifying the most common passenger titles in the Titanic dataset;
- creating survival summaries by sex or passenger class;
- summarizing tuberculosis cases by age group or sex; and
- visualizing categorical distributions using bar plots.

These exercises help reinforce the connection between text cleaning, data reshaping, and exploratory analysis.

5.10 Chapter summary

In this chapter, you learned how to work with character variables and regular expressions using the tidyverse and `stringr`. You cleaned and separated text variables, extracted patterns using regex, created reusable functions, reshaped wide datasets into tidy long-format structures, and interpreted coded variable names.

These techniques are extremely important in real-world data management because many datasets contain poorly structured text variables and encoded information that must be cleaned before analysis can begin.

Chapter 6

Visualization and Advanced Data Cleaning

In previous chapters, we focused on importing, cleaning, reshaping, and joining datasets using the tidyverse. In this chapter, we extend those workflows by introducing exploratory visualization and more advanced data-cleaning techniques commonly used in population health and environmental data analysis.

The overall objective is to prepare several datasets for later analytical work involving cancer mortality, population estimates, geographic correspondence files, and environmental exposure data. Along the way, we will review common challenges such as missing values, inconsistent variable names, duplicate records, invalid dates, and poorly formatted identifiers.

Although the datasets used in this course are simplified for teaching purposes, the workflow closely reflects the types of cleaning and preparation steps encountered in real-world epidemiological and environmental health projects.

The chapter also introduces several important visualization techniques using `ggplot2`, which will help you explore and validate your cleaned datasets.

6.1 Research context and datasets

The exercises in this chapter are motivated by two simplified research questions.

The first analysis investigates age- and sex-specific cancer mortality rates by health region in British Columbia. The second analysis explores whether environmental exposure variables can be linked to mortality data in order to perform a simplified odds-ratio analysis.

In practice, incidence or cancer case data would generally be preferable to mortality data for exposure studies. However, the emphasis of this course is on data cleaning and management rather than formal epidemiological interpretation.

Four datasets will be used throughout the chapter:

- a mortality dataset containing fictitious death records;
- a population dataset containing age- and sex-specific population estimates;

- a geographic correspondence file linking dissemination blocks to health regions; and
- an environmental dataset containing weather variables by postal code.

We begin by loading the required libraries.

```
library(tidyverse)
library(readxl)
library(lubridate)
library(stringr)
```

Next, we import the datasets into R.

```
mort <- read_excel(
  "./Raw Data/deaths_2016.xlsx"
)

pop <- read_csv(
  "./Raw Data/Population_Estimates.csv"
)
```

```
## New names:
## Rows: 51 Columns: 25
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (2): Health Service Delivery Area, Gender dbl (23): ...1, Year, <1, 04-Jan,
## 09-May, 14-Oct, 15-19, 20-24, 25-29, 30-34...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```
corr <- read_csv(
  "./Raw Data/Corr_2016.csv",
  locale = readr::locale(
    encoding = "latin1"
  )
)
```

```
## Rows: 420963 Columns: 6
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): hrname_english, hrname_french
## dbl (4): dbuid2016, csduid2016, hruid2017, dbpop2016
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
env <- read_csv(
  "./Raw Data/Weather_data.csv"
)
```

```
## Rows: 116011 Columns: 8
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): POSTALCODE12
## dbl (7): WTHNRC12_01, WTHNRC12_02, WTHNRC12_03, WTHNRC12_04, WTHNRC12_05, WT...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

6.2 Reviewing datasets before cleaning

Before modifying any dataset, it is important to understand its overall structure. Initial exploratory review often helps identify unexpected variable names, missing values, duplicate records, or formatting inconsistencies.

We first create a list containing all datasets.

```
data_list <- list(
  mort = mort,
  pop = pop,
  corr = corr,
  env = env
)
```

We can then quickly review dataset dimensions.

```
map_dfr(
  data_list,
  ~ tibble(
    rows = nrow(.x),
    columns = ncol(.x)
  ),
  .id = "dataset"
)
```

```
## # A tibble: 4 x 3
##   dataset   rows columns
##   <chr>    <int>   <int>
## 1 mort     60143      13
## 2 pop         51      25
## 3 corr    420963       6
## 4 env     116011       8
```

Column names can also be inspected.

```
map(data_list, names)
```

```
## $mort
##  [1] "ID"                "Sex"               "Cause_of_death"
##  [4] "dbuid2016"         "Location_of_death" "Marital_status"
##  [7] "Postalcode"        "B_year"            "B_month"
## [10] "B_day"             "D_year"            "D_month"
## [13] "D_day"
##
## $pop
##  [1] "...1"                          "Health Service Delivery Area"
##  [3] "Year"                          "Gender"
##  [5] "<1"                            "04-Jan"
##  [7] "09-May"                        "14-Oct"
##  [9] "15-19"                         "20-24"
## [11] "25-29"                         "30-34"
## [13] "35-39"                         "40-44"
## [15] "45-49"                         "50-54"
## [17] "55-59"                         "60-64"
## [19] "65-69"                         "70-74"
## [21] "75-79"                         "80-84"
## [23] "85-89"                         "90+"
## [25] "Total"
##
## $corr
## [1] "dbuid2016"      "csduid2016"     "hruid2017"      "hrname_english"
## [5] "hrname_french"  "dbpop2016"
##
## $env
## [1] "POSTALCODE12" "WTHNRC12_01"  "WTHNRC12_02"  "WTHNRC12_03"  "WTHNRC12_04"
## [6] "WTHNRC12_05"  "WTHNRC12_06"  "WTHNRC12_07"
```

Finally, we summarize missing values across datasets.

```
map(
  data_list,
  ~ map_int(.x, ~ sum(is.na(.x)))
)
```

```
## $mort
##                ID               Sex    Cause_of_death         dbuid2016
##                 0                13                 0                 4
## Location_of_death    Marital_status        Postalcode            B_year
##                 0                 0                 6                24
```

```
##           B_month             B_day            D_year           D_month
##                26                26                 0                 0
##             D_day
##                 0
##
## $pop
##                          ...1 Health Service Delivery Area
##                             0                             0
##                          Year                        Gender
##                             0                             0
##                            <1                        04-Jan
##                             0                             0
##                        09-May                        14-Oct
##                             0                             0
##                         15-19                         20-24
##                             0                             0
##                         25-29                         30-34
##                             0                             0
##                         35-39                         40-44
##                             0                             0
##                         45-49                         50-54
##                             0                             0
##                         55-59                         60-64
##                             0                             0
##                         65-69                         70-74
##                             0                             0
##                         75-79                         80-84
##                             0                             0
##                         85-89                           90+
##                             0                             0
##                         Total
##                             0
##
## $corr
##      dbuid2016     csduid2016      hruid2017 hrname_english  hrname_french
##              0              0              0              0              0
##      dbpop2016
##             35
##
## $env
## POSTALCODE12  WTHNRC12_01  WTHNRC12_02  WTHNRC12_03  WTHNRC12_04  WTHNRC12_05
##            0            0            0            0            0            0
##  WTHNRC12_06  WTHNRC12_07
##            0            0
```

Early exploratory review is one of the most important stages of data cleaning because it helps identify potential problems before they propagate through the analysis workflow.

Always inspect datasets immediately after import. Many analytical problems originate from unnoticed import issues, incorrect variable types, or unexpected missing values.

6.3 Cleaning and reshaping population data

The population dataset is currently stored in a wide format, where age groups appear as separate columns. Although this structure may be useful for spreadsheets, it is less convenient for visualization and analysis.

A tidy long-format structure is generally easier to work with in R.

We first reshape the dataset using `pivot_longer()`.

```
pop_long <- pop %>%

  select(
    -any_of(c("X1", "Total"))
  ) %>%

  pivot_longer(
    cols = -any_of(
      c(
        "Year",
        "Gender",
        "Health Service Delivery Area"
      )
    ),
    names_to = "Age",
    values_to = "Population"
  ) %>%

  mutate(
    Age = case_when(
      Age == "04-Jan" ~ "01-04",
      Age == "09-May" ~ "05-09",
      Age == "14-Oct" ~ "10-14",
      TRUE ~ Age
    )
  )

pop_long
```

```
## # A tibble: 1,071 x 5
##    `Health Service Delivery Area`  Year Gender Age   Population
##    <chr>                          <dbl> <chr>  <chr>      <dbl>
##  1 British Columbia                2016 M      ...1           0
##  2 British Columbia                2016 M      <1         22997
```

```
##  3 British Columbia                2016 M      01-04      93211
##  4 British Columbia                2016 M      05-09     121341
##  5 British Columbia                2016 M      10-14     119586
##  6 British Columbia                2016 M      15-19     140451
##  7 British Columbia                2016 M      20-24     170968
##  8 British Columbia                2016 M      25-29     159609
##  9 British Columbia                2016 M      30-34     162416
## 10 British Columbia                2016 M      35-39     155936
## # i 1,061 more rows
```

Some variable names are unnecessarily long or contain spaces, so we simplify them.

```
pop_long <- pop_long %>%

  rename(
    HSDA = `Health Service Delivery Area`
  ) %>%

  select(-Year)

pop_long
```

```
## # A tibble: 1,071 x 4
##    HSDA             Gender Age   Population
##    <chr>            <chr>  <chr>      <dbl>
##  1 British Columbia M      ...1           0
##  2 British Columbia M      <1         22997
##  3 British Columbia M      01-04      93211
##  4 British Columbia M      05-09     121341
##  5 British Columbia M      10-14     119586
##  6 British Columbia M      15-19     140451
##  7 British Columbia M      20-24     170968
##  8 British Columbia M      25-29     159609
##  9 British Columbia M      30-34     162416
## 10 British Columbia M      35-39     155936
## # i 1,061 more rows
```

For this example, we focus on total population counts only.

```
pop_total <- pop_long %>%

  filter(Gender == "T")

pop_total
```

```
## # A tibble: 357 x 4
```

```
##    HSDA             Gender Age   Population
##    <chr>            <chr>  <chr>      <dbl>
##  1 British Columbia T      ...1           0
##  2 British Columbia T      <1         44757
##  3 British Columbia T      01-04     181511
##  4 British Columbia T      05-09     234118
##  5 British Columbia T      10-14     232010
##  6 British Columbia T      15-19     272677
##  7 British Columbia T      20-24     327129
##  8 British Columbia T      25-29     318998
##  9 British Columbia T      30-34     329242
## 10 British Columbia T      35-39     313564
## # i 347 more rows
```

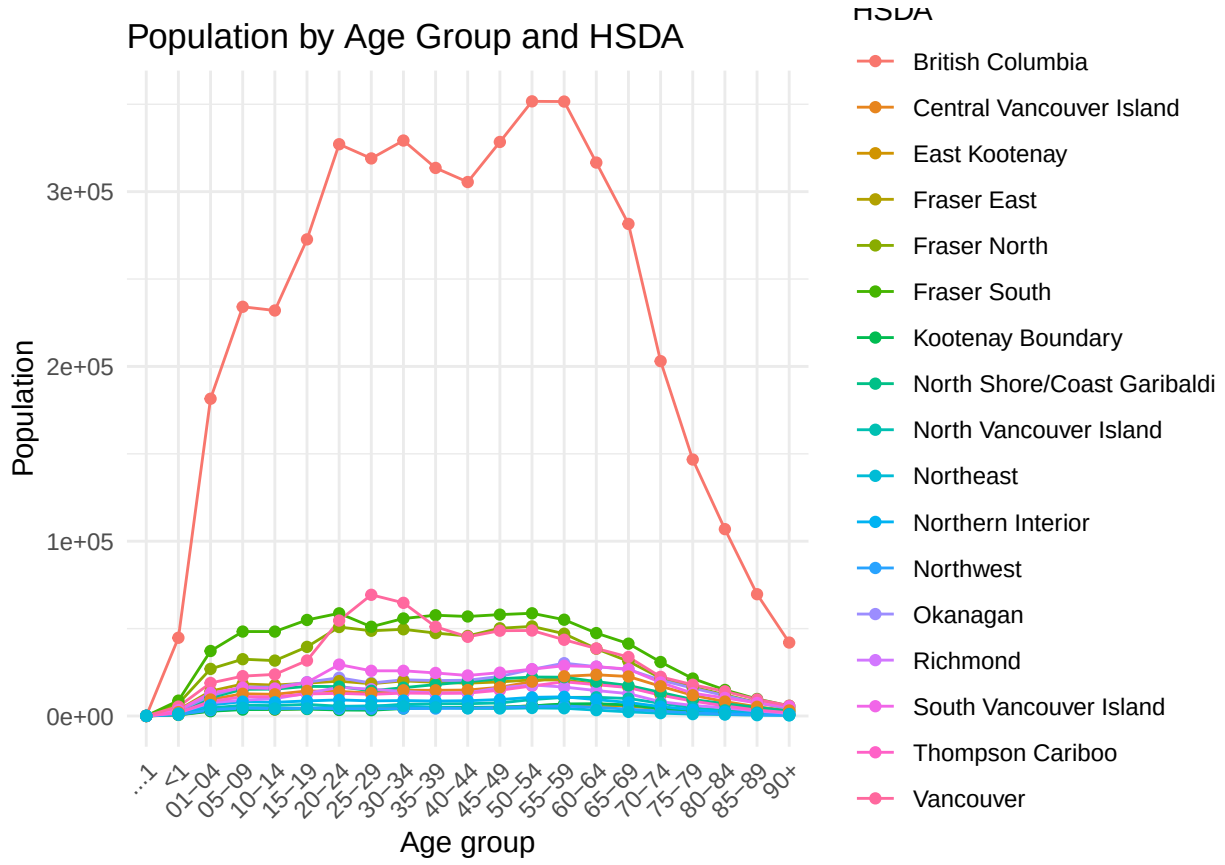
6.4 Visualizing population data

Visualization is an important part of exploratory analysis because it helps reveal trends, inconsistencies, and unusual values that may not be obvious in tables alone.

The following line plot displays population counts across age groups by HSDA.

```
pop_total %>%

  ggplot(
    aes(
      x = Age,
      y = Population,
      group = HSDA,
      colour = HSDA
    )
  ) +

  geom_line() +

  geom_point() +

  labs(
    title = "Population by Age Group and HSDA",
    x = "Age group",
    y = "Population",
    colour = "HSDA"
  ) +

  theme_minimal() +

  theme(
    axis.text.x = element_text(
```

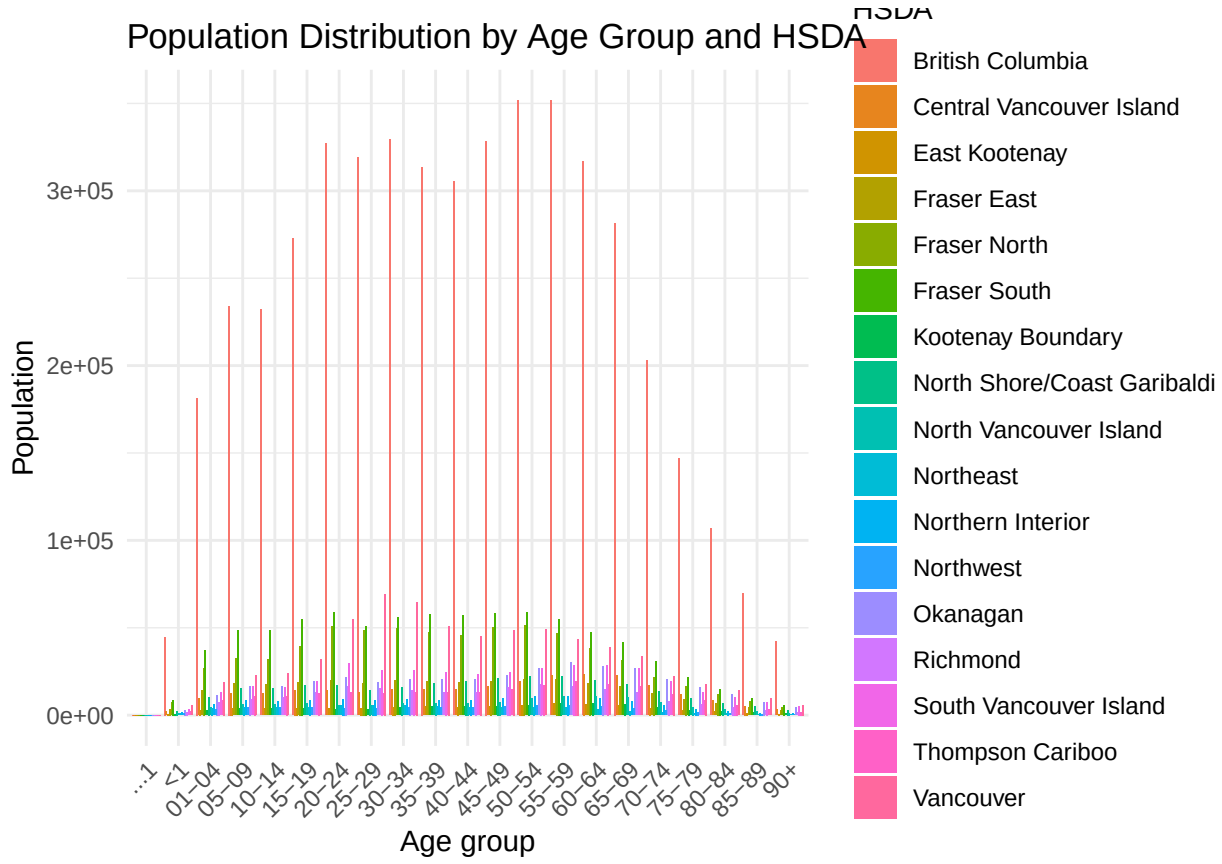
```
      angle = 45,
      hjust = 1
    )
  )
```



Because age groups are categorical rather than continuous, a bar plot may sometimes be easier to interpret.

```
pop_total %>%

  ggplot(
    aes(
      x = Age,
      y = Population,
      fill = HSDA
    )
  ) +

  geom_col(position = "dodge") +

  labs(
    title = "Population Distribution by Age Group and HSDA",
    x = "Age group",
```

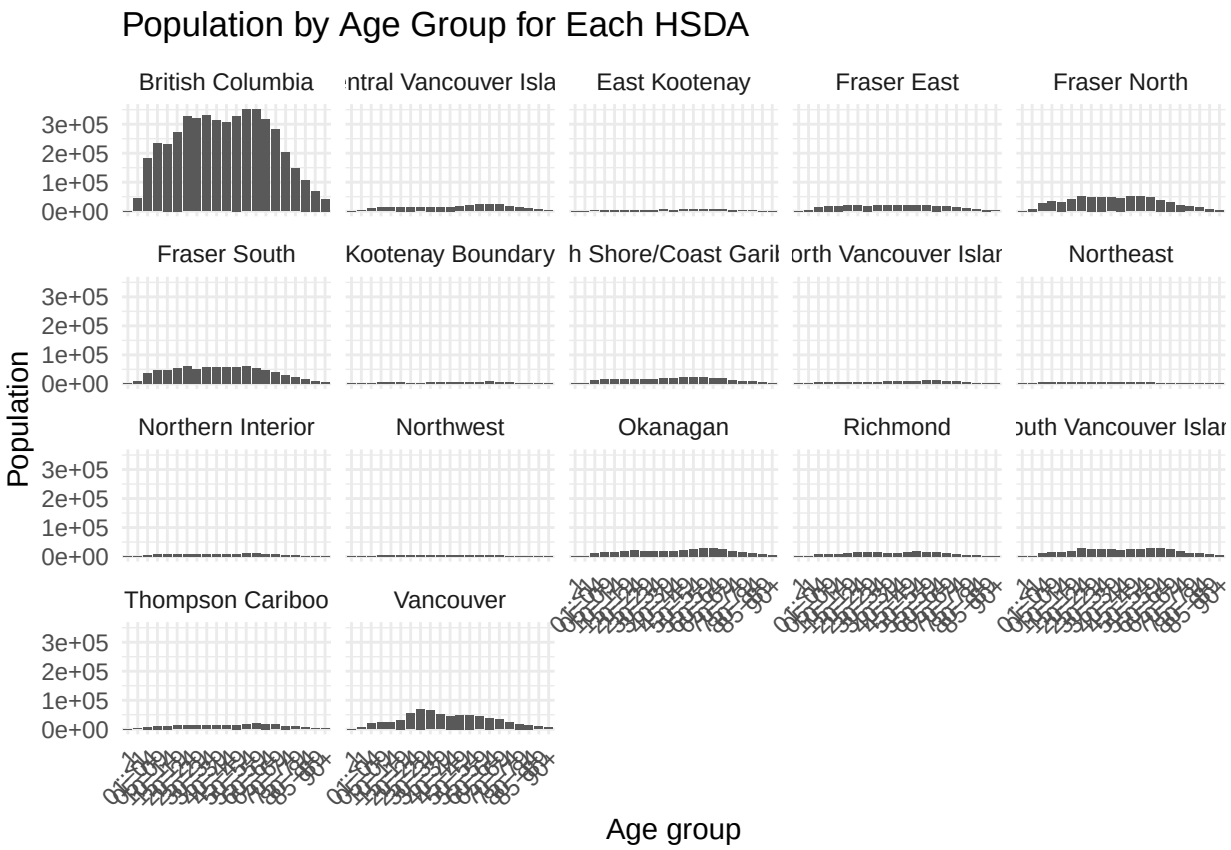
```
    y = "Population",
    fill = "HSDA"
  ) +

  theme_minimal() +

  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
```



When plots become crowded, faceting can improve readability.

```
pop_total %>%

  ggplot(
    aes(
      x = Age,
      y = Population
    )
  ) +
```

```
  geom_col() +

  facet_wrap(~ HSDA) +

  labs(
    title = "Population by Age Group for Each HSDA",
    x = "Age group",
    y = "Population"
  ) +

  theme_minimal() +

  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
```



6.5 Identifying join keys

Before combining datasets, it is important to identify variables that can serve as join keys.

```
map(data_list, names)
```

```
## $mort
##  [1] "ID"                "Sex"               "Cause_of_death"
##  [4] "dbuid2016"         "Location_of_death" "Marital_status"
##  [7] "Postalcode"        "B_year"            "B_month"
## [10] "B_day"             "D_year"            "D_month"
## [13] "D_day"
##
## $pop
##  [1] "...1"                          "Health Service Delivery Area"
##  [3] "Year"                          "Gender"
##  [5] "<1"                            "04-Jan"
##  [7] "09-May"                        "14-Oct"
##  [9] "15-19"                         "20-24"
## [11] "25-29"                         "30-34"
## [13] "35-39"                         "40-44"
## [15] "45-49"                         "50-54"
## [17] "55-59"                         "60-64"
## [19] "65-69"                         "70-74"
## [21] "75-79"                         "80-84"
## [23] "85-89"                         "90+"
## [25] "Total"
##
## $corr
## [1] "dbuid2016"      "csduid2016"     "hruid2017"      "hrname_english"
## [5] "hrname_french"  "dbpop2016"
##
## $env
## [1] "POSTALCODE12" "WTHNRC12_01"  "WTHNRC12_02"  "WTHNRC12_03"  "WTHNRC12_04"
## [6] "WTHNRC12_05"  "WTHNRC12_06"  "WTHNRC12_07"
```

Possible join paths include:

- joining mortality data to the correspondence file using `dbuid2016`;
- joining population data to geographic information using HSDA-related variables; and
- linking environmental data to mortality records using postal codes.

Before performing joins, always inspect whether the key variables are unique and consistently formatted.

Incorrect joins are one of the most common causes of duplicate records and analytical errors in real-world projects.

6.6 Cleaning mortality data

We now begin cleaning the mortality dataset.

Variables not required for the current analysis are removed.

```
rates <- mort %>%

  select(
    -c(
      Location_of_death,
      Marital_status
    )
  )

rates
```

```
## # A tibble: 60,143 x 11
##         ID Sex   Cause_of_death dbuid2016 Postalcode B_year B_month B_day D_year
##      <dbl> <chr> <chr>               <dbl> <chr>       <dbl>   <dbl> <dbl>  <dbl>
##  1 1000001 M     D47              1.31e10 E1A1E9       1943       7     8   2016
##  2 1000002 F     C34              4.81e10 T2E0T2       1943       5    22   2016
##  3 1000003 M     C26              3.52e10 M4J1L1       1953       8     9   2016
##  4 1000004 M     C16              2.47e10 H9H1B4       1923       6    14   2016
##  5 1000005 M     C22              2.46e10 J2W2B6       1931       8     8   2016
##  6 1000006 F     C50              3.53e10 L8H2K1       1933      11     4   2016
##  7 1000007 F     C85              3.54e10 N9Y3X5       1918       5    20   2016
##  8 1000008 F     C56              3.53e10 L8H7G2       1972       8    18   2016
##  9 1000009 F     C64              2.47e10 H9P2B3       1922       7    23   2016
## 10 1000010 M     C22              4.81e10 T5M1G2       1953       2     7   2016
## # i 60,133 more rows
## # i 2 more variables: D_month <dbl>, D_day <dbl>
```

Duplicate records should also be identified and reviewed.

```
rates_dup <- rates %>%

  count(ID) %>%

  filter(n > 1)

rates_dup
```

```
## # A tibble: 23 x 2
##         ID     n
##      <dbl> <int>
##  1 1000170     2
```

```
##  2 1000662     2
##  3 1001349     2
##  4 1001352     2
##  5 1004287     2
##  6 1004618     2
##  7 1004869     2
##  8 1005245     2
##  9 1008199     2
## 10 1012433     2
## # i 13 more rows
```

```
rates %>%

  filter(ID %in% rates_dup$ID)
```

```
## # A tibble: 47 x 11
##         ID Sex   Cause_of_death dbuid2016 Postalcode B_year B_month B_day D_year
##      <dbl> <chr> <chr>               <dbl> <chr>       <dbl>   <dbl> <dbl>  <dbl>
##  1 1000170 F     C91              1.21e10 B4V1T1       1922       7    25   2016
##  2 1000170 F     C91              1.21e10 B4V1T1       1922       7    25   2016
##  3 1000662 M     C34              2.47e10 J7P4H7       1965       3    29   2016
##  4 1000662 M     C34              2.47e10 J7P4H7       1965       3    29   2016
##  5 1001349 M     C16              2.44e10 G9A5H9       1961       4     6   2016
##  6 1001349 M     C16              2.44e10 G9A5H9       1961       4     6   2016
##  7 1001352 M     C24              4.80e10 T1B3E3       1956      12    27   2016
##  8 1001352 M     C24              4.80e10 T1B3E3       1956      12    27   2016
##  9 1004287 M     C34              3.53e10 N4S2E1       1949       2    18   2016
## 10 1004287 M     C34              3.53e10 N4S2E1       1949       2    18   2016
## # i 37 more rows
## # i 2 more variables: D_month <dbl>, D_day <dbl>
```

After inspection, duplicates can be removed.

```
rates <- rates %>%

  distinct()
```

6.7 Handling missing and inconsistent values

Missing values and data-entry problems are common in administrative datasets.

We begin by examining birth year distributions.

```
rates %>%

  ggplot(
```

```
    aes(x = B_year)
  ) +

  geom_histogram(binwidth = 5) +

  labs(
    title = "Distribution of Birth Year",
    x = "Birth year",
    y = "Count"
  ) +

  theme_minimal()
```

```
## Warning: Removed 24 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

Some birth years before 1905 are likely data-entry errors. For this exercise, we assume these values should belong to the twentieth century.

```
rates <- rates %>%

  mutate(
```

```
    B_year = if_else(
      B_year < 1905,
      B_year + 100,
      B_year
    ),

    B_month = if_else(
      is.na(B_month),
      6,
      B_month
    ),

    B_day = if_else(
      is.na(B_day),
      15,
      B_day
    )
  )
```

We also inspect death months and days.

```
rates %>%
  count(D_month)
```

```
## # A tibble: 17 x 2
##    D_month     n
##      <dbl> <int>
##  1       1  5088
##  2       2  4677
##  3       3  5056
##  4       4  4880
##  5       5  5108
##  6       6  5015
##  7       7  5147
##  8       8  5195
##  9       9  4927
## 10      10  5038
## 11      11  4913
## 12      12  5070
## 13      16     1
## 14      17     1
## 15      18     2
## 16      19     1
## 17      21     1
```

```
rates <- rates %>%

  mutate(
    D_month = if_else(
      D_month > 12,
      6,
      D_month
    )
  )

rates %>%
  count(D_month)
```

```
## # A tibble: 12 x 2
##    D_month     n
##      <dbl> <int>
##  1       1  5088
##  2       2  4677
##  3       3  5056
##  4       4  4880
##  5       5  5108
##  6       6  5021
##  7       7  5147
##  8       8  5195
##  9       9  4927
## 10      10  5038
## 11      11  4913
## 12      12  5070
```

```
rates %>%
  count(D_day)
```

```
## # A tibble: 31 x 2
##    D_day     n
##    <dbl> <int>
##  1     1  1954
##  2     2  1996
##  3     3  1972
##  4     4  2022
##  5     5  1929
##  6     6  1960
##  7     7  1986
##  8     8  1966
##  9     9  1956
## 10    10  1903
## # i 21 more rows
```

```
rates %>%
  count(B_day)
```

```
## # A tibble: 31 x 2
##    B_day     n
##    <dbl> <int>
##  1     1  1979
##  2     2  2029
##  3     3  1949
##  4     4  1937
##  5     5  1916
##  6     6  1877
##  7     7  2001
##  8     8  1960
##  9     9  1942
## 10    10  1995
## # i 21 more rows
```

6.8 Creating dates and calculating age

The `lubridate` package simplifies date handling in R.

We create dates of birth and death and then calculate age.

```
rates <- rates %>%

  mutate(
    DOB = as_date(
      str_c(
        B_year,
        B_month,
        B_day,
        sep = "-"
      )
    ),

    DOD = as_date(
      str_c(
        D_year,
        D_month,
        D_day,
        sep = "-"
      )
    ),

    Age = year(DOD) - year(DOB),
```

```
    Age = if_else(
      Age < 0,
      0.5,
      as.numeric(Age)
    )
  )
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `DOB = as_date(str_c(B_year, B_month, B_day, sep = "-"))`.
## Caused by warning:
## !  6 failed to parse.
```

```
range(rates$Age, na.rm = TRUE)
```

```
## [1]   0 111
```

Remaining missing values can then be inspected.

```
map_int(
  rates,
  ~ sum(is.na(.x))
)
```

```
##             ID            Sex Cause_of_death      dbuid2016     Postalcode
##              0             13              0              4              6
##         B_year        B_month          B_day         D_year        D_month
##             24              0              0              0              0
##          D_day            DOB            DOD            Age
##              0             30              0             30
```

```
rates %>%

  filter(
    if_any(
      everything(),
      is.na
    )
  )
```

```
## # A tibble: 53 x 14
##         ID Sex   Cause_of_death dbuid2016 Postalcode B_year B_month B_day D_year
##      <dbl> <chr> <chr>              <dbl> <chr>       <dbl>   <dbl> <dbl>  <dbl>
##  1 1000185 F     C85              5.94e10 V4V2G1         NA       6    15   2016
##  2 1000345 <NA>  C19              5.92e10 V3W1S7       1964       4    22   2016
```

```
##  3 1001139 F     C50             5.95e10 V0J0B5         NA       6    15   2016
##  4 1001262 M     C25             5.92e10 <NA>         1952       1    25   2016
##  5 1001402 F     C34             5.94e10 V1E1V2         NA       7    11   2016
##  6 1001696 <NA>  C34             5.92e10 V8T4P6       1931       8     6   2016
##  7 1001964 M     C61             5.92e10 <NA>         1948       1    22   2016
##  8 1002150 M     C26             5.91e10 V2P6S3         NA       6    15   2016
##  9 1003446 F     C34             5.92e10 V8T4E5         NA       6    15   2016
## 10 1004221 M     C34             5.92e10 V8V4V6         NA       6    15   2016
## # i 43 more rows
## # i 5 more variables: D_month <dbl>, D_day <dbl>, DOB <date>, DOD <date>,
## #   Age <dbl>
```

Records missing essential identifiers are removed.

```
rates <- rates %>%

  drop_na(
    B_year,
    dbuid2016
  )
```

6.9 Imputing missing values

For demonstration purposes, missing sex values are randomly imputed.

```
set.seed(123)

missing_sex_index <- is.na(rates$Sex)

missing_sex_count <- sum(missing_sex_index)

rates$Sex <- as.character(rates$Sex)

rates$Sex[missing_sex_index] <- sample(
  c("M", "F"),
  size = missing_sex_count,
  replace = TRUE
)

rates$Sex <- as.factor(rates$Sex)

rates %>%
  count(Sex)
```

```
## # A tibble: 2 x 2
##   Sex       n
```

```
##   <fct> <int>
## 1 F     28617
## 2 M     31475
```

Although this approach is useful for teaching purposes, real-world imputation should be performed much more carefully.

Random imputation can introduce bias into analyses. Always document assumptions and cleaning decisions clearly.

6.10 Working with ICD-10 codes

Cause-of-death variables often contain coded medical classifications.

We separate ICD-10 codes into letter and numeric components.

```
rates <- rates %>%

  extract(
    Cause_of_death,
    into = c("letter", "number"),
    regex = "([A-Z]+)([0-9]+)",
    remove = FALSE
  ) %>%

  mutate(
    number = as.numeric(number)
  )

rates %>%
  count(letter)
```

```
## # A tibble: 2 x 2
##   letter     n
##   <chr>  <int>
## 1 C      58916
## 2 D       1176
```

Cancer-related mortality records are identified using ICD-10 codes beginning with `"C"`.

```
rates <- rates %>%

  filter(letter == "C")
```

We then classify cancer categories and age groups.

```
rates <- rates %>%

  mutate(
    CauseCat = case_when(
      number == 50 ~ 1,
      number == 61 ~ 2,
      number %in% c(33, 34) ~ 3,
      number %in% 18:21 ~ 4,
      TRUE ~ 5
    ),

    CauseCat_label = case_when(
      CauseCat == 1 ~ "Breast cancer",
      CauseCat == 2 ~ "Prostate cancer",
      CauseCat == 3 ~ "Lung cancer",
      CauseCat == 4 ~ "Colorectal cancer",
      CauseCat == 5 ~ "Other cancer"
    ),

    AgeCat = cut(
      Age,
      breaks = c(
        0, 10, 20, 30, 40,
        50, 60, 70, 80, 90, 130
      ),
      include.lowest = TRUE,
      right = FALSE
    )
  )

rates %>%
  count(CauseCat_label)
```

```
## # A tibble: 5 x 2
##   CauseCat_label        n
##   <chr>             <int>
## 1 Breast cancer      4003
## 2 Colorectal cancer  6564
## 3 Lung cancer       14914
## 4 Other cancer      30328
## 5 Prostate cancer    3107
```

```
rates %>%
  count(AgeCat)
```

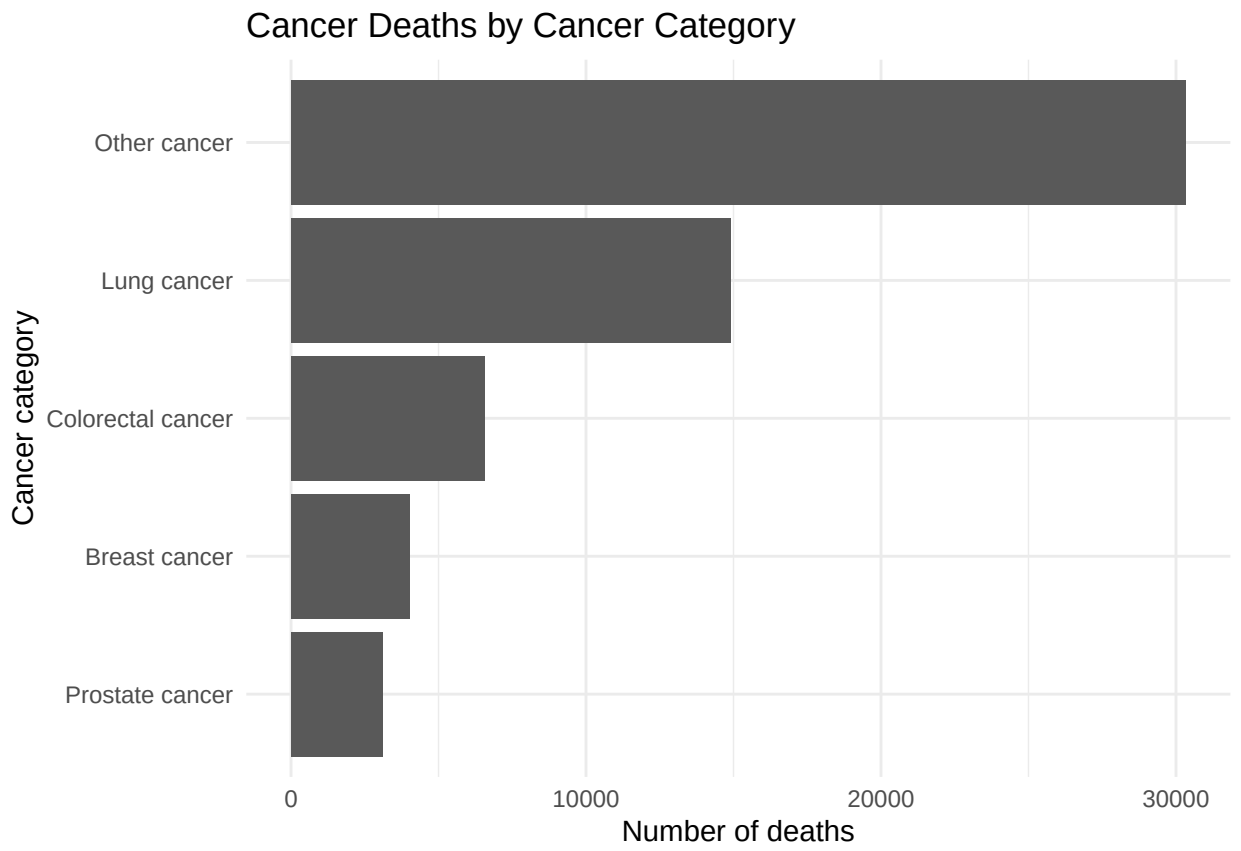
```
## # A tibble: 11 x 2
```

```
##    AgeCat       n
##    <fct>    <int>
##  1 [0,10)      66
##  2 [10,20)     81
##  3 [20,30)    148
##  4 [30,40)    459
##  5 [40,50)   1522
##  6 [50,60)   5866
##  7 [60,70)  13086
##  8 [70,80)  16495
##  9 [80,90)  15520
## 10 [90,130]  5667
## 11 <NA>         6
```

6.11 Visualizing cleaned mortality data

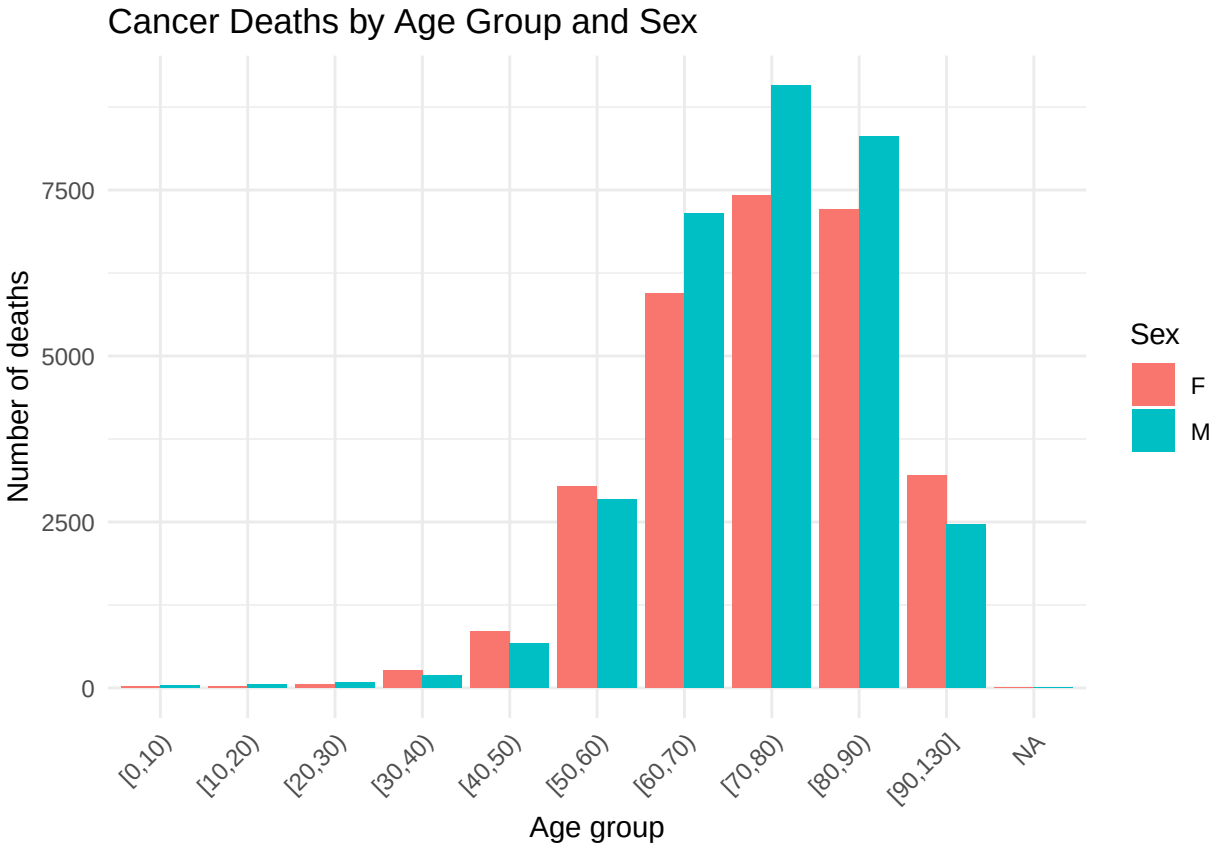
After cleaning the mortality data, we can explore the distributions visually.

```
rates %>%

  count(CauseCat_label) %>%

  ggplot(
    aes(
      x = reorder(CauseCat_label, n),
      y = n
    )
  ) +

  geom_col() +

  coord_flip() +

  labs(
    title = "Cancer Deaths by Cancer Category",
    x = "Cancer category",
    y = "Number of deaths"
  ) +

  theme_minimal()
```




```
rates %>%

  count(AgeCat, Sex) %>%

  ggplot(
    aes(
      x = AgeCat,
      y = n,
      fill = Sex
    )
  ) +

  geom_col(position = "dodge") +

  labs(
    title = "Cancer Deaths by Age Group and Sex",
    x = "Age group",
    y = "Number of deaths",
    fill = "Sex"
  ) +

  theme_minimal() +
```

```
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
```



6.12 Saving cleaned datasets

Cleaned outputs should always be saved separately from raw data.

We first create an output folder if it does not already exist.

```
dir.create(
  "./Outputs",
  showWarnings = FALSE
)
```

We then save the cleaned mortality data.

```
write_csv(
  rates,
  "./Outputs/Cancer_rates.csv"
)
```

Environmental exposure data can also be cleaned and saved.

```
env_clean <- env %>%

  select(
    POSTALCODE12,
    WTHNRC12_04,
    WTHNRC12_05
  ) %>%

  rename(
    Postal_code = POSTALCODE12,
    max_temp = WTHNRC12_04,
    min_temp = WTHNRC12_05
  ) %>%

  mutate(
    exposure = if_else(
      max_temp < 10 &
      min_temp < -3,
      1,
      0
    )
  )

env_clean
```

```
## # A tibble: 116,011 x 4
##    Postal_code max_temp min_temp exposure
##    <chr>          <dbl>    <dbl>    <dbl>
##  1 V0C1E0          2.91    -8.12        1
##  2 V0C1W0          3.92    -7.65        1
##  3 V0C2X0          4.01    -7.93        1
##  4 V0C2Z0          4.32    -6.89        1
##  5 V0T1W0          4.33    -3.65        1
##  6 V0W1A0          4.51    -4.92        1
##  7 V0C1L0          4.58    -6.14        1
##  8 V0J1K0          4.71    -5.34        1
##  9 V0B1A1          4.91    -4.42        1
## 10 V0B1T6          4.91    -4.42        1
## # i 116,001 more rows
```

```
write_csv(
  env_clean,
  "./Outputs/exposures.csv"
)
```

6.13 Joining datasets and validating postal codes

The mortality dataset can now be joined to the correspondence file.

```
analysis1 <- inner_join(
  rates,
  corr,
  by = "dbuid2016"
)

analysis1
```

```
## # A tibble: 48,682 x 24
##        ID Sex   Cause_of_death letter number dbuid2016 Postalcode B_year B_month
##     <dbl> <fct> <chr>          <chr>   <dbl>     <dbl> <chr>        <dbl>   <dbl>
##  1 1.00e6 F     C34            C          34   4.81e10 T2E0T2        1943       5
##  2 1.00e6 M     C26            C          26   3.52e10 M4J1L1        1953       8
##  3 1.00e6 M     C16            C          16   2.47e10 H9H1B4        1923       6
##  4 1.00e6 M     C22            C          22   2.46e10 J2W2B6        1931       8
##  5 1.00e6 F     C50            C          50   3.53e10 L8H2K1        1933      11
##  6 1.00e6 F     C85            C          85   3.54e10 N9Y3X5        1918       5
##  7 1.00e6 F     C56            C          56   3.53e10 L8H7G2        1972       8
##  8 1.00e6 F     C64            C          64   2.47e10 H9P2B3        1922       7
##  9 1.00e6 M     C22            C          22   4.81e10 T5M1G2        1953       2
## 10 1.00e6 F     C50            C          50   1.21e10 B3A4B2        1978      11
## # i 48,672 more rows
## # i 15 more variables: B_day <dbl>, D_year <dbl>, D_month <dbl>, D_day <dbl>,
## #   DOB <date>, DOD <date>, Age <dbl>, CauseCat <dbl>, CauseCat_label <chr>,
## #   AgeCat <fct>, csduid2016 <dbl>, hruid2017 <dbl>, hrname_english <chr>,
## #   hrname_french <chr>, dbpop2016 <dbl>
```

Postal codes should also be standardized before linkage.

```
rates <- rates %>%

  mutate(
    Postalcode_clean =
      Postalcode %>%

      str_to_upper() %>%
```

```
      str_replace_all(
        "\\s+",
        ""
      )
  )
```

We can then identify postal codes that do not follow the expected Canadian format.

```
postal_pattern <-
  "^(?!.*[DFIOQU])[A-VXY][0-9][A-Z][0-9][A-Z][0-9]$"

postal_issues <- rates %>%

  filter(
    is.na(
      str_match(
        Postalcode_clean,
        postal_pattern
      )
    )
  ) %>%

  select(
    ID,
    Postalcode,
    Postalcode_clean
  )
```

```
## Warning: Using one column matrices in `filter()` or `filter_out()` was deprecated in
## dplyr 1.1.0.
## i Please use one dimensional logical vectors instead.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
postal_issues
```

```
## # A tibble: 38 x 3
##         ID Postalcode Postalcode_clean
##      <dbl> <chr>      <chr>
##  1 1000103 V2A6X      V2A6X
##  2 1000695 V5S2HY     V5S2HY
##  3 1000760 V6Y1ML     V6Y1ML
##  4 1000896 VOC1HO     VOC1HO
##  5 1001210 V0ROB8     V0ROB8
##  6 1001262 <NA>       <NA>
```

```
##  7 1001964 <NA>       <NA>
##  8 1003432 V2SOA4     V2SOA4
##  9 1003907 VOB1HO     VOB1HO
## 10 1004334 VOB1G8     VOB1G8
## # i 28 more rows
```

Regular-expression validation is an extremely useful technique for detecting formatting problems in identifiers such as postal codes, phone numbers, or IDs.

6.14 Final assignment guidance

As you continue building analytical workflows, begin organizing your work into separate exploratory data analysis documents.

Each analysis file should clearly explain:

- the datasets being used;
- the cleaning and validation steps performed;
- assumptions made during cleaning;
- how missing values were handled; and
- how outputs were generated.

A well-documented workflow is just as important as the analysis itself.

6.15 Chapter summary

In this chapter, you expanded your data management workflow by combining advanced cleaning techniques with exploratory visualization. You reviewed datasets, reshaped population data, created visualizations, identified join keys, cleaned mortality and environmental datasets, handled missing values, validated identifiers, and created cleaned outputs for later analysis.

These workflows form the foundation of reproducible analytical projects and closely resemble the types of preparation steps required in real-world public health and environmental data analysis.

Chapter 7

Exploratory Analysis Project

In previous chapters, we focused on importing, cleaning, reshaping, joining, and visualizing datasets using the tidyverse. This chapter brings those individual skills together into a more complete exploratory analysis workflow.

The purpose of exploratory analysis is not to produce final analytical results, but rather to understand the structure and quality of the data before formal analysis begins. A well-prepared exploratory analysis document should allow a colleague, manager, or future researcher to understand the datasets being used, the cleaning decisions that were made, and the reasoning behind those decisions.

In practice, exploratory analysis is one of the most important stages of a data project because many analytical problems can be identified early through careful inspection of the data.

By the end of this chapter, you should be able to organize multiple datasets into a reproducible workflow, summarize dataset structures, evaluate missing values, create reusable functions, and communicate cleaning decisions clearly using R Markdown.

7.1 Loading libraries and importing datasets

We begin by loading the libraries used throughout the exploratory analysis workflow.

```
library(tidyverse)
library(readxl)
library(lubridate)
```

The datasets used in this project include mortality records, population estimates, geographic correspondence files, and environmental exposure data.

```
mort <- read_excel(
  "./Raw Data/deaths_2016.xlsx"
)

pop <- read_csv(
```

```
  "./Raw Data/Population_Estimates.csv"
)

corr <- read_csv(
  "./Raw Data/Corr_2016.csv",
  locale = readr::locale(
    encoding = "latin1"
  )
)

env <- read_csv(
  "./Raw Data/Weather_data.csv"
)
```

When working with multiple datasets, it is often helpful to place them into a single list structure. This makes it easier to apply functions repeatedly across all datasets using tools from the `purrr` package.

```
data_list <- list(
  mortality = mort,
  population = pop,
  correspondence = corr,
  environment = env
)
```

7.2 Reviewing dataset structure

A good exploratory analysis begins with understanding the basic structure of each dataset. Important questions include:

- How many rows and columns are present?
- What variables are available?
- Are there obvious missing values?
- Are the variable types correct?
- Are there unusual or unexpected values?

The `map()` function allows us to apply the same operation across all datasets efficiently.

For example, we can review dataset dimensions.

```
map(data_list, dim)

map(data_list, nrow)

map(data_list, ncol)
```

We can also inspect variable names.

```
map(data_list, names)
```

Functions such as `glimpse()` and `summary()` are also useful during exploratory review.

```
map(data_list, glimpse)

map(data_list, summary)
```

These exploratory checks often reveal problems early in the workflow, including inconsistent variable naming, missing values, incorrect data types, or unusual coding structures.

One of the most common mistakes in data analysis is beginning formal analysis before fully understanding the structure and quality of the data.

7.3 Working with missing values

Missing data are extremely common in real-world datasets. Before deciding how to handle missing values, it is important to understand how frequently they occur and whether they appear systematically within specific variables.

Instead of repeatedly writing the same code for each dataset, we can create a reusable function.

```
count_missing <- function(data) {

  data %>%

    summarize(
      across(
        everything(),
        ~ sum(is.na(.))
      )
    )
}
```

This function calculates the number of missing values for every variable in a dataset.

We can then apply the function across all datasets.

```
map(data_list, count_missing)
```

Reusable functions improve efficiency, reduce duplicated code, and make workflows easier to maintain.

It is also important to remember that missing values are not always random. In some cases, missingness may indicate systematic collection problems, unavailable information, or meaningful absence of data.

Never remove missing values automatically without first understanding why they are missing and whether the missingness may influence the analysis.

7.4 Duplicate records and cleaning decisions

In addition to missing values, duplicate records should also be investigated during exploratory analysis.

For example:

```
mort %>%

  count(ID) %>%

  filter(n > 1)
```

Duplicate records can arise from data-entry errors, repeated imports, or incorrect joins. Identifying these issues early helps prevent inflated counts and inaccurate results later in the workflow.

As cleaning decisions are made, they should be documented clearly in plain language. Good documentation is one of the defining features of reproducible analysis.

For example, instead of writing only code comments such as:

```
# fixed dates
```

it is better to explain the reasoning more clearly:

```
# Birth years before 1905 were assumed to be data-entry errors
# and adjusted forward by 100 years.
```

Clear explanations help future users understand not only *what* changes were made, but also *why* those changes were considered appropriate.

7.5 Exploratory visualization

Visualization is another important component of exploratory analysis. Simple plots can quickly reveal outliers, skewed distributions, missing categories, or unusual relationships between variables.

For example, histograms are useful for examining continuous variables.

```
mort %>%

  ggplot(
    aes(x = B_year)
  ) +

  geom_histogram(binwidth = 5) +

  labs(
```

```
    title = "Distribution of Birth Year",
    x = "Birth year",
    y = "Count"
  ) +

  theme_minimal()
```

Bar plots are often useful for categorical variables.

```
mort %>%

  count(Sex) %>%

  ggplot(
    aes(x = Sex, y = n)
  ) +

  geom_col() +

  labs(
    title = "Distribution of Sex",
    x = "Sex",
    y = "Count"
  ) +

  theme_minimal()
```

Exploratory visualizations do not need to be highly polished. Their primary purpose is to help you understand the data and identify potential problems.

7.6 Organizing an exploratory analysis report

A well-structured exploratory analysis document should guide the reader logically through the datasets and cleaning workflow.

Although the exact structure may vary between projects, most exploratory analysis reports include the following components:

1. A short description of the project purpose and research questions.
2. A description of the datasets being used.
3. Summary information about dataset dimensions and structure.
4. Explanations of important variables.
5. Missing-value summaries and duplicate checks.
6. Cleaning and transformation decisions.
7. Initial visualizations and descriptive summaries.
8. A brief discussion of key findings and remaining data issues.

9. Proposed next steps for future analysis.

The goal is not to create a perfect final report, but rather to create a clear and reproducible record of the exploratory workflow.

7.7 Practice activity

Create a new R Markdown document called:

```
exploratory_analysis.Rmd
```

Use the structure introduced in this chapter to build a short exploratory analysis report for the datasets used throughout the course.

Your report should include:

- at least one table summarizing dataset dimensions;
- one missing-value summary;
- at least one visualization;
- a short explanation of cleaning decisions; and
- clear comments describing major analytical steps.

As you write the document, focus on clarity and reproducibility. Imagine that another analyst will need to understand and continue your work in the future.

7.8 Chapter summary

In this chapter, you combined many of the skills introduced earlier in the course into a more complete exploratory analysis workflow. You organized datasets into reusable structures, summarized dimensions and variable information, evaluated missing values, created reusable functions, checked for duplicate records, and produced simple exploratory visualizations.

Most importantly, you practiced documenting cleaning decisions and analytical reasoning in a way that supports reproducibility and collaboration. Exploratory analysis is not simply a preliminary step before “real analysis”; it is a critical stage that shapes the quality and reliability of all later results.

Chapter 8

Storyboarding and Reporting

Data analysis is not only about writing code and generating outputs. An important part of any analytical workflow is communicating the results clearly to other people. Even a technically correct analysis can become ineffective if the final report is confusing, disorganized, or difficult to interpret.

This chapter introduces the idea of *storyboarding*, which is the process of planning the structure and flow of an analysis before fully developing the report. Storyboarding helps organize ideas, identify the main analytical message, and create a logical structure that guides the reader through the analysis.

In many professional environments, analytical reports are written for audiences who may not have strong technical backgrounds. Managers, clients, policy teams, and decision-makers often need a concise explanation of the results and their implications rather than detailed programming logic. A well-structured report helps bridge the gap between technical analysis and practical communication.

By the end of this chapter, you should be able to organize analytical results into a coherent narrative, distinguish technical notes from reader-focused explanations, and prepare a polished R Markdown report that communicates findings clearly and professionally.

8.1 Why storyboarding is important

Many beginners begin writing reports by immediately producing plots and tables without first deciding what story the analysis should communicate. This often leads to reports that feel disorganized, repetitive, or difficult to follow.

Storyboarding helps solve this problem by encouraging analysts to think about the structure of the report before focusing on formatting details or extensive coding.

A good storyboard answers several important questions:

- What is the main purpose of the analysis?
- Who is the intended audience?
- What are the most important findings?
- What evidence supports those findings?
- What limitations or uncertainties should be acknowledged?
- What should happen next?

Thinking about these questions early helps create reports that are more focused and easier to interpret.

One of the most common reporting mistakes is including every result produced during the analysis. Effective reports focus on the results that are most relevant to the research question or decision-making process.

8.2 Organizing the analytical narrative

Most analytical reports follow a similar general structure. Although the exact format may vary depending on the audience and project, the report should guide the reader logically from the problem statement to the conclusions.

A simple storyboard structure often includes the following components:

Section	Purpose
Background	Explain why the analysis is being performed
Data	Describe the datasets and variables used
Cleaning and preparation	Explain important cleaning decisions and assumptions
Analysis	Present summaries, comparisons, or calculations
Visualization	Show key figures or tables
Findings	Summarize the main results
Limitations	Discuss uncertainty or missing information
Next steps	Recommend future analysis or actions

This structure creates a clear analytical flow that helps readers understand not only the results, but also how those results were produced.

In practice, storyboarding often begins with a rough outline written in plain language before any code or plots are finalized.

For example, before creating visualizations, you might first write:

> We want to determine whether cancer mortality rates differ by age group and health region. Before calculating rates, we need to examine population distributions and evaluate whether age structures differ across regions.

Writing the analytical purpose in plain language first often helps clarify what figures and summaries are actually needed.

8.3 Writing interpretation instead of only showing output

One of the most important parts of reporting is interpretation. Tables and plots should rarely appear without explanation.

After each major figure or summary table, include several sentences explaining:

- what the output shows;
- why the result matters;
- what patterns or trends are important; and
- what limitations should be considered.

For example:

> The plot shows that population counts vary substantially across age groups and Health Service Delivery Areas. Older age groups appear more concentrated in some regions than others, suggesting that age standardization may be important before comparing mortality rates between regions.

This type of interpretation helps connect the visualization to the broader analytical purpose.

Without interpretation, readers are often left to guess what conclusions they should draw from the output.

A good analytical report explains the meaning of the results, not only the code used to produce them.

8.4 Separating technical details from reader-focused explanations

Not all readers require the same level of technical detail. Technical notes are important for reproducibility, but too much technical explanation can interrupt the flow of the report for non-technical audiences.

In many cases, it is useful to separate:

- technical implementation details; and
- reader-focused interpretation.

For example, instead of writing:

> We used `pivot_longer()` to reshape the data from wide to long format.

a more reader-focused explanation might be:

> The population data were reorganized into a tidy structure so that age groups could be analyzed and visualized consistently.

The technical code remains visible in the R Markdown document, but the written explanation focuses on the analytical purpose rather than the programming mechanics.

This approach creates reports that are both reproducible and easier to read.

8.5 Building a polished R Markdown report

R Markdown is particularly useful for reporting because it combines narrative text, code, tables, and visualizations within a single reproducible document.

A polished report should include:

- a clear title and introduction;
- consistent formatting and headings;
- explanatory text between code chunks;
- readable plots and tables;
- interpretation of important outputs; and
- a concise summary of findings.

When preparing figures, readability should be prioritized over decoration. Axes should be labeled clearly, titles should describe the figure meaningfully, and overcrowded plots should be simplified whenever possible.

For example, instead of a title such as:

```
Plot 1
```

a more informative title would be:

```
Cancer Mortality by Age Group and Sex
```

Clear figure titles improve communication and reduce ambiguity.

Similarly, comments and narrative explanations should focus on helping readers understand the workflow rather than simply describing the code syntax.

8.6 Example interpretation workflow

Suppose a visualization shows population counts by age group and health region.

A weak interpretation might be:

> This is a bar plot of population by age.

A stronger interpretation would be:

> The visualization shows that population distributions differ across age groups and health regions. Some regions appear to have relatively older populations, which may influence comparisons of crude mortality rates. This suggests that age-specific or age-standardized analyses may be more appropriate than direct comparisons of overall mortality counts.

The second explanation connects the visualization to analytical reasoning and future decision-making.

8.7 Practice activity

Choose one visualization created earlier in the course and write a short interpretation paragraph discussing:

1. what the visualization shows;
2. why the result matters;
3. one limitation of the figure or dataset; and
4. what analysis or investigation should follow next.

Focus on writing clearly for a reader who may not have a strong technical background.

8.8 Chapter summary

In this chapter, you learned how storyboarding can improve the organization and clarity of analytical reports. You explored how to structure an analysis into a logical narrative, interpret visualizations in plain language, separate technical implementation details from reader-focused explanations, and prepare polished R Markdown reports.

Strong analytical reporting is not only about producing correct results. It is also about communicating those results clearly, transparently, and persuasively to the intended audience.